

Интеграция ИИ в три концепции Smart Money (SMC): Ордер-блок (OB), слом структуры (BOS) и гэп справедливой стоимости (FVG)

1 Hlomohang John Borotho 207

Содержание

1. Введение
2. Обзор системы
3. Начало работы
4. Интеграция с MQL5
5. Бэктест
6. Заключение

Введение

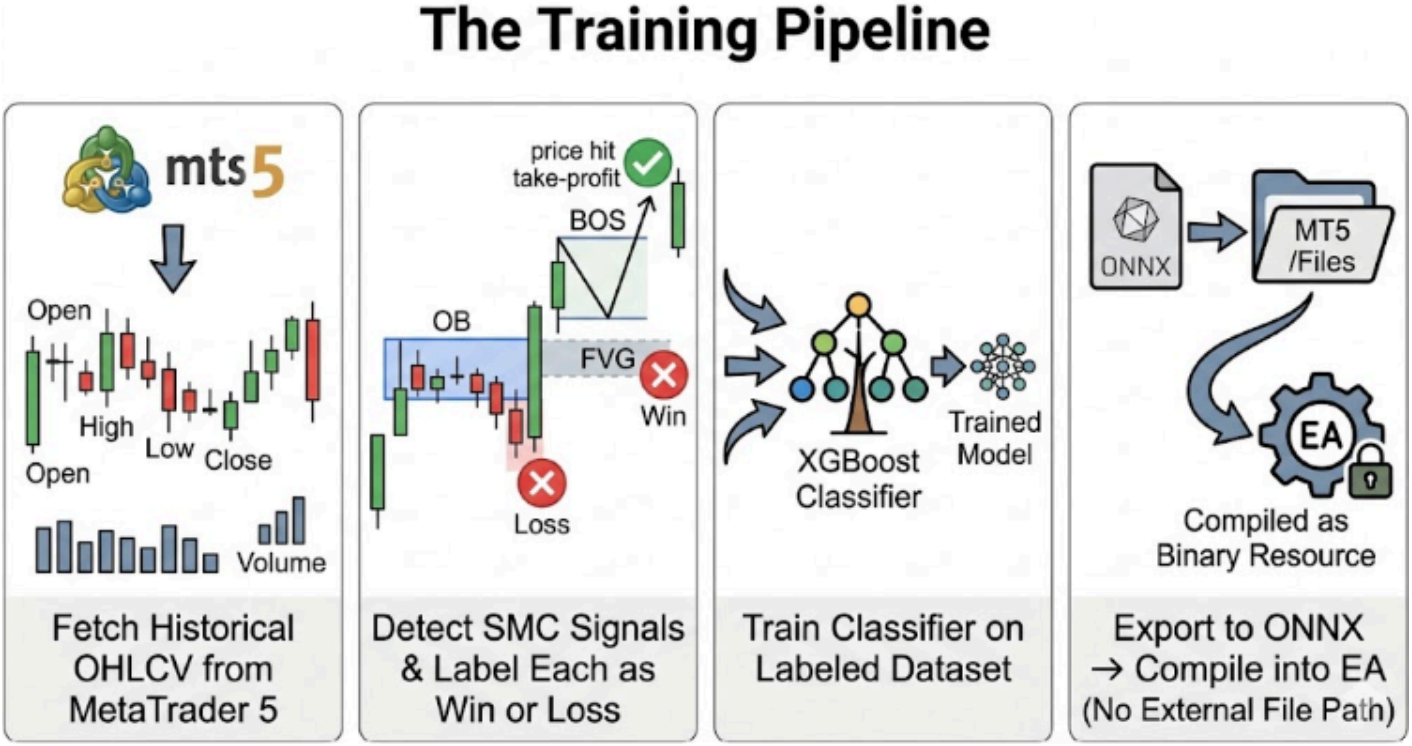
Торговля с использованием концепций Smart Money обеспечивает структурированный подход к анализу рынка, однако сама по себе структура не гарантирует стабильности результатов. Даже при наличии чётко определённых сигналов ордер-блоков (OB), гэпов справедливой стоимости (FVG) и слома структуры (BOS) трейдеры по-прежнему сталкиваются с задачей решить, каким сетапам доверять, а какие игнорировать. Не каждый откат к ордер-блоку (OB) приводит к продолжению движения, не каждое заполнение гэпа справедливой стоимости (FVG) вызывает разворот, и не каждый слом структуры (BOS) отражает настоящий рыночный сдвиг. Этот непрерывный процесс фильтрации приводит к появлению эмоциональной предвзятости, нерешительности и непоследовательности, особенно при торговле такими динамичными инструментами, как XAUUSD, где рыночная ситуация может резко меняться даже в течение одной свечи.

В этой статье это ограничение устраняется за счёт внедрения слоя машинного обучения в SMC-советник из предыдущей темы. С помощью Python мы обучаем бинарный классификатор XGBoost на исторических данных по инструменту XAUUSD на таймфрейме H1. Затем мы экспортируем его в формат ONNX и компилируем в советник как двоичный ресурс. При появлении сигнала советник формирует вектор из 12 признаков, описывающий рыночную структуру, импульс, время торговой сессии и контекст тренда, а затем оценивает его с помощью модели ИИ перед исполнением. Система также выводит на панель в режиме реального времени информацию об источнике сигнала, уверенности ИИ и силе тренда, а трейлинг-стоп с учётом волатильности динамически управляет открытыми позициями.

Обзор системы

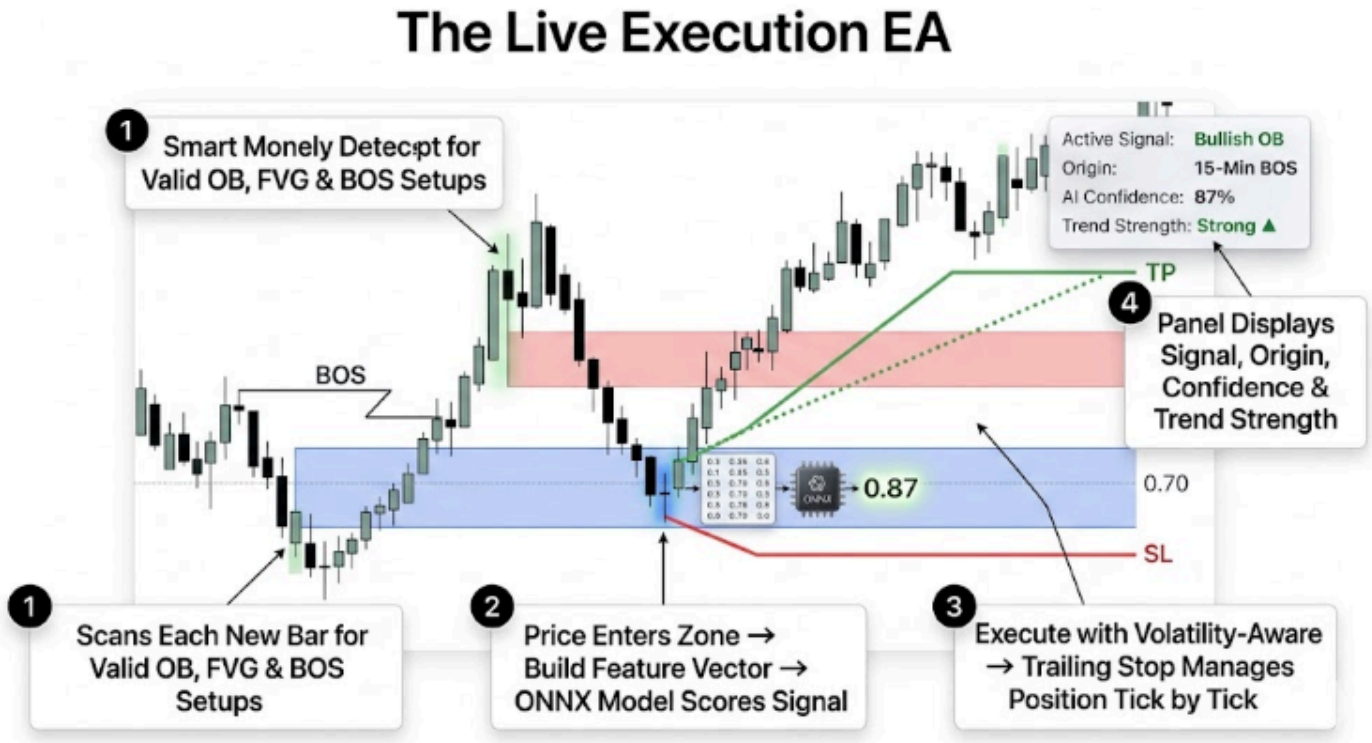
Система построена на двух отдельных, но тесно связанных между собой слоях. Первый уровень представляет собой пайплайн обучения на языке Python. Он извлекает данные OHLCV из MetaTrader 5, обнаруживает сигналы SMC, помечает каждое событие как прибыльное или убыточное на основе TP/SL

и обучает классификатор XGBoost на полученном наборе данных. По завершении обучения модель экспортируется в формат ONNX и сохраняется непосредственно в папке Files MetaTrader 5. После этого MetaEditor компилирует её в советник в виде двоичного ресурса, то есть модель встраивается в исполняемый файл и не требует указания пути к файлу во время выполнения.



Модель обучается офлайн в Python, а затем встраивается непосредственно в исполняемый файл советника. Во время выполнения путь к файлу не требуется — «мозг» находится внутри робота

Второй слой — это сам советник MQL5, который обрабатывает всё, что происходит на графике. На каждом новом баре он ищет допустимые торговые сетапы ордер-блоков (OB), гэпов справедливой стоимости (FVG) и сломов структуры (BOS), используя ту же логику обнаружения, что и во время обучения. Когда цена входит в зону, советник формирует вектор признаков на основе текущего рыночного контекста и передаёт его во встроенную модель ONNX для оценки. Если оценка достигает минимального порога уверенности, сделка исполняется со стоп-лоссом и тейк-профитом, учитывающими волатильность. Затем трейлинг-стоп управляет открытой позицией тик за тиком, фиксируя прибыль по мере движения цены. На панели графика отображаются активный сигнал, его источник, оценка уверенности ИИ и текущая сила тренда. Это обеспечивает прозрачность решений, принимаемых системой.



Советник выполняет функции глаз, мозга и рук: выявляет торговые сетапы, оценивает их с помощью встроенной модели ONNX, открывает сделки и показывает трейдеру, почему было принято каждое решение

Начало работы

```
from datetime import datetime
import sys
import warnings

import pandas as pd
import pytz

warnings.filterwarnings("ignore")

# ============================================================================
#  CONFIG — edit these values
# ============================================================================
SYMBOL      = "XAUUSD"        # exact symbol name as shown in MT5 Market Watch
TIMEFRAME   = "H1"              # M1 M5 M15 M30 H1 H4 D1
DATE_FROM   = datetime(2020, 1,  1)   # start of historical range
DATE_TO     = datetime(2026, 1,  1)   # end   of historical range (exclusive)
OUTPUT_CSV  = "XAUUSD_H1.csv"         # output filename (saved in working directory)
TIMEZONE    = "Etc/UTC"               # keep UTC — pipeline expects UTC timestamps
# ============================================================================

# Timeframe string -> MT5 constant name
TF_MAP = {
    "M1":  "TIMEFRAME_M1",
    "M5":  "TIMEFRAME_M5",
    "M15": "TIMEFRAME_M15",
    "M30": "TIMEFRAME_M30",
    "H1":  "TIMEFRAME_H1",
    "H4":  "TIMEFRAME_H4",
```

```
    "D1":  "TIMEFRAME_D1",
}

def fetch():
    try:
        import MetaTrader5 as mt5
    except ImportError:
        sys.exit("[ERROR] MetaTrader5 package not installed.\n"
                 "        Run:  pip install MetaTrader5")

    print(f"MetaTrader5 package  v{mt5.__version__}  by {mt5.__author__}")

    # ── Initialise ──────────────────────────────────────────────────────────
    if not mt5.initialize():
        sys.exit(f"[ERROR] mt5.initialize() failed: {mt5.last_error()}")
    print(f"[MT5]  Connected - build {mt5.version()}")

    # ── Select symbol ───────────────────────────────────────────────────────
    if not mt5.symbol_select(SYMBOL, True):
        mt5.shutdown()
        sys.exit(f"[ERROR] Symbol '{SYMBOL}' not found in Market Watch.\n"
                 "        Check the exact name (e.g. XAUUSD vs XAUUSD.m vs XAUUSDm)")

    info = mt5.symbol_info(SYMBOL)
    point = info.point if info else 0.01
    print(f"[MT5]  Symbol: {SYMBOL}  Point: {point}  Digits: {info.digits if info else

    # ── Validate timeframe ──────────────────────────────────────────────────
    tf_attr = TF_MAP.get(TIMEFRAME.upper())
    if tf_attr is None:
        mt5.shutdown()
        sys.exit(f"[ERROR] Unknown timeframe '{TIMEFRAME}'. Choose from: {list(TF_MAP)}
    tf = getattr(mt5, tf_attr)

    # ── Build UTC-aware date range ──────────────────────────────────────────
    tz       = pytz.timezone(TIMEZONE)
    utc_from = tz.localize(DATE_FROM)
    utc_to   = tz.localize(DATE_TO)
    print(f"[MT5]  Requesting {TIMEFRAME} bars from {utc_from.date()} to {utc_to.date()

    # ── Fetch ───────────────────────────────────────────────────────────────
    rates = mt5.copy_rates_range(SYMBOL, tf, utc_from, utc_to)
    mt5.shutdown()

    if rates is None or len(rates) == 0:
        sys.exit("[ERROR] No bars returned. Possible causes:\n"
                 "  - Date range has no data for this symbol on this broker\n"
                 "  - Symbol requires a different name (try without .m suffix)\n"
                 "  - MT5 history for this period not downloaded yet\n"
                 "    (Open the chart in MT5 and scroll back to force download)")

    # ── Build DataFrame ─────────────────────────────────────────────────────
    df = pd.DataFrame(rates)
    df["time"] = pd.to_datetime(df["time"], unit="s", utc=True)
    df.set_index("time", inplace=True)

    # Rename tick_volume -> volume, drop spread column if present
```

```
    df.rename(columns={"tick_volume": "volume", "real_volume": "real_vol"}, inplace=Tru
    keep = [c for c in ["open", "high", "low", "close", "volume"] if c in df.columns]
    df = df[keep].astype(float)

    # ── Quality report ──────────────────────────────────────────────────────
    n_bars     = len(df)
    date_start = df.index[0].strftime("%Y-%m-%d %H:%M")
    date_end   = df.index[-1].strftime("%Y-%m-%d %H:%M")
    price_min  = df["close"].min()
    price_max  = df["close"].max()
    atr_proxy  = (df["high"] - df["low"]).mean()
    gaps       = df.index.to_series().diff().dropna()
    expected   = gaps.mode()[0]
    gap_bars   = (gaps > expected * 1.5).sum()

    print(f"\n{'='*55}")
    print(f"  Data Quality Report")
    print(f"{'='*55}")
    print(f"  Bars fetched   : {n_bars:,}")
    print(f"  Date range     : {date_start}  ->  {date_end}")
    print(f"  Price range    : {price_min:.2f}  ->  {price_max:.2f}")
    print(f"  Mean ATR (H-L) : {atr_proxy:.2f}")
    print(f"  Missing bars   : {gap_bars:,}  (weekend/holiday gaps expected)")
    print(f"{'='*55}\n")

    if n_bars < 1000:
        print(f"[WARN]  Only {n_bars} bars fetched. Consider extending the date range.\
              "        Training needs at least 5,000+ bars for meaningful results.")

    # ── Save ────────────────────────────────────────────────────────────────
    df.to_csv(OUTPUT_CSV)
    print(f"[OK]    Saved {n_bars:,} bars to '{OUTPUT_CSV}'")
    print(f"        File size: {pd.io.common.get_handle(OUTPUT_CSV,'r').handle.seek(0,2)
          f"{round(Path(OUTPUT_CSV).stat().st_size / 1024, 1)} KB")
    print(f"\nNext step: set csv_file = '{OUTPUT_CSV}' in the training pipeline.")
    return df


# ── Entrypoint ──────────────────────────────────────────────────────────────
from pathlib import Path

if __name__ == "__main__":
    fetch()
else:
    # When imported or run as a Jupyter cell, execute immediately
    fetch()
```

Этот скрипт создаёт пайплайн извлечения исторических данных, используемый для обучения торговой модели ИИ. Он напрямую подключается к MetaTrader 5 с помощью Python, выбирает указанный инструмент и таймфрейм, а также загружает исторические рыночные данные OHLCV за настраиваемый диапазон дат в UTC. Полученные данные преобразуются в очищенный DataFrame библиотеки Pandas, индексируются по временным меткам и стандартизируются в единообразный формат, подходящий для машинного обучения. Скрипт также выполняет облегчённый анализ качества, выводя общее количество баров, ценовой диапазон, среднюю волатильность свечей и потенциальные пропуски баров, вызванные выходными днями или недоступностью исторических данных.

Результат:

```
MetaTrader5 package  v5.0.5572  by MetaQuotes Ltd.
[MT5]  Connected — build (500, 5833, '25 Apr 2026')
[MT5]  Symbol: XAUUSD  Point: 0.001  Digits: 3
[MT5]  Requesting H1 bars from 2020-01-01 to 2026-01-01 ...

============================================================
  Data Quality Report
============================================================
  Bars fetched   : 13,584
  Date range     : 2023-09-04 01:00  ->  2025-12-31 23:00
  Price range    : 1815.03  ->  4546.69
  Mean ATR (H-L) : 8.17
  Missing bars   : 709  (weekend/holiday gaps expected)
============================================================

[OK]   Saved 13,584 bars to 'XAUUSD_H1.csv'
       File size: 873.9 KB

Next step: set csv_file = 'XAUUSD_H1.csv' in the training pipeline.
```

```python
import argparse
import os
import sys
import warnings
from pathlib import Path

import numpy as np
import pandas as pd

warnings.filterwarnings("ignore")

# ═══════════════════════════════════════════════════════════════════════════
#  NOTEBOOK CONFIG — edit these values (ignored when run from CLI)
# ═══════════════════════════════════════════════════════════════════════════
NOTEBOOK_CONFIG = dict(
    # ── Data source ──────────────────────────────────────────────────────────
    # "csv"      : load from CSV_FILE produced by fetch_historical_data.py  ✅ recommend
    # "mt5"      : pull bars live from MT5 (MT5 must be open)
    # "synthetic": random-walk data for quick pipeline tests
    data_source = "csv",
    csv_file    = "XAUUSD_H1.csv",   # path to the CSV file

    # ── MT5 live-pull settings (used only when data_source="mt5") ────────────
    symbol    = "XAUUSD.m",
    timeframe = "H1",
    bars      = 50000,

    # ── Trade parameters — MUST match EA inputs exactly ──────────────────────
    # XAUUSD point = 0.01, so:
    #   sl_pts=3500  ->  SL distance = 3500 × 0.01 = $35.00
    #   tp_pts=7500  ->  TP distance = 7500 × 0.01 = $75.00
    sl_pts    = 3500,
    tp_pts    = 7500,

    # ── SMC detection parameters — MUST match EA inputs exactly ──────────────
    swing_len = 5,        # SwingPeriod in EA
    fvg_min   = 3,        # FVG_MinPoints in EA
    fib_lvl   = 61.8,   # Fib_Trade_lvls in EA
```

```
    # ── Output ──────────────────────────────────────────────────────────────
    out_dir   = ".",      # where to save smc_features.csv (audit log)

    # Direct path to MT5 Files folder -- model is saved here automatically.
    # MT5 -> File -> Open Data Folder -> MQL5 -> Files
    mt5_files_path = r'C:\Users\...\AppData\Roaming\MetaQuotes\Terminal\....\MQL5\Files
)
# ==============================================================================


# ------------------------------------------------------------------------------
# Symbol point-size lookup  (mirrors MQL5 _Point behaviour)
# ------------------------------------------------------------------------------
SYMBOL_POINT = {
    "XAUUSD": 0.01,
    "XAGUSD": 0.001,
    "USDJPY": 0.001,
    "EURJPY": 0.001,
    "GBPJPY": 0.001,
    "US30":   0.01,
    "NAS100": 0.01,
    "SPX500": 0.01,
    "USTEC":  0.01,
}

def get_point(symbol: str) -> float:
    sym = symbol.upper()
    for key, val in SYMBOL_POINT.items():
        if key in sym:
            return val
    return 0.00001    # default: standard 5-digit FX
```

В этом разделе определяется базовый уровень конфигурации для всего пайплайна обучения ИИ. Мы определяем, откуда будут поступать рыночные данные: из ранее экспортированного CSV-файла, через активное подключение к MetaTrader 5 или из синтетических данных случайного блуждания для целей тестирования. Кроме того, в конфигурации централизованно хранятся все параметры стратегии, которые должны оставаться синхронизированными с советником. К ним относятся расстояния SL и TP, настройки обнаружения свингов, чувствительность гэпа справедливой стоимости (FVG), а также уровни коррекции Фибоначчи. Сохраняя эти значения в одном структурированном блоке конфигурации, мы гарантируем, что среда обучения на Python и логика исполнения MQL5 работают на основе одинаковых рыночных допущений.

Кроме того, мы реализуем систему сопоставления размера пункта для инструмента, которая повторяет поведение _Point, используемое внутри MQL5. Это позволяет пайплайну правильно интерпретировать ценовые расстояния для разных инструментов, таких как XAUUSD, индексы и валютные пары с JPY. Вспомогательная функция get_point() автоматически определяет правильный размер пункта для выбранного инструмента. Это гарантирует, что расчёты SL и TP, конструирование признаков и реконструкция сигналов остаются полностью согласованными с советником как во время обучения, так и при работе в реальном времени.

```
# ------------------------------------------------------------------------------
# Config resolution (Jupyter-safe argparse)
# ------------------------------------------------------------------------------
def get_config():
    """
```

```
    Returns a plain Namespace from NOTEBOOK_CONFIG when inside Jupyter,
    or from CLI arguments when run from a terminal.
    """
    running_in_jupyter = (
        "ipykernel_launcher" in sys.argv[0]
        or any(a.startswith("--f=") or a == "-f" for a in sys.argv[1:])
    )

    if running_in_jupyter:
        print("[Config] Jupyter detected -- using NOTEBOOK_CONFIG.")
        print("[Config] Edit the NOTEBOOK_CONFIG dict at the top of this file.\n")
        return argparse.Namespace(**NOTEBOOK_CONFIG)

    p = argparse.ArgumentParser(description="SMC XGBoost -> ONNX trainer")
    p.add_argument("--data_source", default="csv",
                   choices=["csv", "mt5", "synthetic"])
    p.add_argument("--csv_file",    default="XAUUSD_H1.csv")
    p.add_argument("--symbol",      default="XAUUSD.m")
    p.add_argument("--timeframe",   default="H1")
    p.add_argument("--bars",        type=int,   default=50000)
    p.add_argument("--sl_pts",      type=int,   default=3500)
    p.add_argument("--tp_pts",      type=int,   default=7500)
    p.add_argument("--swing_len",   type=int,   default=5)
    p.add_argument("--fvg_min",     type=int,   default=3)
    p.add_argument("--fib_lvl",     type=float, default=61.8)
    p.add_argument("--out_dir",     default=".")
    p.add_argument("--mt5_files_path", default="",
                   help="Direct path to MQL5/Files folder")
    return p.parse_args()


# ---------------------------------------------------------------------------
# Data loaders
# ---------------------------------------------------------------------------
def load_csv(csv_file: str) -> pd.DataFrame:
    """
    Load the CSV produced by fetch_historical_data.py.
    Handles both UTC-aware and naive timestamps.
    Expected columns: time (index), open, high, low, close, volume
    """
    if not Path(csv_file).exists():
        sys.exit(
            f"[ERROR] CSV file not found: '{csv_file}'\n"
            "        Run fetch_historical_data.py first to generate it."
        )

    df = pd.read_csv(csv_file, index_col=0, parse_dates=True)

    # Normalise index to UTC-aware
    if df.index.tz is None:
        df.index = pd.to_datetime(df.index, utc=True)
    else:
        df.index = df.index.tz_convert("UTC")

    df.index.name = "time"

    # Keep only OHLCV; rename tick_volume if present
```

```
    df.rename(columns={"tick_volume": "volume"}, inplace=True)
    required = ["open", "high", "low", "close"]
    missing  = [c for c in required if c not in df.columns]
    if missing:
        sys.exit(f"[ERROR] CSV is missing columns: {missing}\n"
                 f"        Found columns: {list(df.columns)}")

    if "volume" not in df.columns:
        df["volume"] = 1.0   # placeholder if volume absent

    df = df[["open", "high", "low", "close", "volume"]].astype(float)
    df.dropna(inplace=True)
    df.sort_index(inplace=True)

    date_start = df.index[0].strftime("%Y-%m-%d")
    date_end   = df.index[-1].strftime("%Y-%m-%d")
    atr_proxy  = (df["high"] - df["low"]).mean()

    print(f"[CSV]   Loaded {len(df):,} bars from '{csv_file}'")
    print(f"        Date range : {date_start}  ->  {date_end}")
    print(f"        Price range: {df['close'].min():.2f}  ->  {df['close'].max():.2f}")
    print(f"        Mean H-L   : {atr_proxy:.4f}  ({atr_proxy:.2f} for XAUUSD-style)\n")
    return df


def load_mt5_bars(symbol: str, tf_str: str, n: int) -> pd.DataFrame:
    try:
        import MetaTrader5 as mt5
    except ImportError:
        sys.exit("MetaTrader5 package not installed.  pip install MetaTrader5")

    if not mt5.initialize():
        sys.exit(f"mt5.initialize() failed: {mt5.last_error()}")

    tf = getattr(mt5, f"TIMEFRAME_{tf_str}", None)
    if tf is None:
        mt5.shutdown()
        sys.exit(f"Unknown timeframe '{tf_str}'")

    rates = mt5.copy_rates_from_pos(symbol, tf, 0, n)
    mt5.shutdown()

    if rates is None or len(rates) == 0:
        sys.exit("No data returned from MT5. Check symbol name and date availability.")

    df = pd.DataFrame(rates)
    df["time"] = pd.to_datetime(df["time"], unit="s", utc=True)
    df.rename(columns={"tick_volume": "volume"}, inplace=True)
    df.set_index("time", inplace=True)
    df = df[["open", "high", "low", "close", "volume"]].astype(float)
    print(f"[MT5]   Loaded {len(df):,} bars of {symbol} {tf_str}")
    return df


def make_synthetic_bars(n: int = 20000, symbol: str = "XAUUSD",
                        timeframe: str = "H1") -> pd.DataFrame:
    """Realistic random-walk data - for quick pipeline tests only."""
```

```
    np.random.seed(42)
    sym = symbol.upper()
    freq_map = {"M1":"1min","M5":"5min","M15":"15min","M30":"30min",
                "H1":"1h","H4":"4h","D1":"1D"}
    freq = freq_map.get(timeframe, "1h")
    dates = pd.date_range("2020-01-01", periods=n, freq=freq, tz="UTC")

    if   "XAU" in sym: base, step, noise, wick = 1900.0, 0.80, 2.50, 4.00
    elif "XAG" in sym: base, step, noise, wick = 24.0,   0.05, 0.15, 0.25
    elif "JPY" in sym: base, step, noise, wick = 130.0,  0.05, 0.15, 0.25
    elif "US30" in sym: base, step, noise, wick = 34000., 50., 120., 200.
    else:               base, step, noise, wick = 1.1,  3e-4, 5e-4, 8e-4

    close     = base + np.cumsum(np.random.normal(0, step, n))
    close     = np.maximum(close, base * 0.5)
    body      = np.abs(np.random.normal(0, noise, n))
    wu        = np.abs(np.random.normal(0, wick,  n))
    wd        = np.abs(np.random.normal(0, wick,  n))
    direction = np.sign(np.random.normal(0, 1, n))
    open_     = close - direction * body
    high      = np.maximum(close, open_) + wu
    low       = np.minimum(close, open_) - wd
    volume    = np.random.randint(500, 8000, n).astype(float)

    df = pd.DataFrame({"open": open_, "high": high, "low": low,
                       "close": close, "volume": volume}, index=dates)
    print(f"[Synth] Generated {n:,} synthetic {symbol} {timeframe} bars "
          f"({close.min():.2f} - {close.max():.2f})")
    return df
```

В этом разделе создаётся инфраструктура конфигурации и загрузки данных для пайплайна обучения ИИ. Сначала мы реализуем совместимый с Jupyter механизм разрешения конфигурации, который автоматически определяет, запущен ли скрипт в среде ноутбука или из командной строки. При запуске в Jupyter пайплайн загружает все параметры непосредственно из словаря NOTEBOOK_CONFIG, что упрощает проведение экспериментов и отладку. При запуске из терминала система переключается на argparse и принимает параметры времени выполнения, такие как выбор инструмента, таймфрейм, настройки стоп-лосса, уровни Фибоначчи, пути вывода и конфигурация источника данных. Такая двухрежимная архитектура позволяет одному и тому же пайплайну обучения единообразно работать в ноутбуках, скриптах и автоматизированных рабочих процессах без необходимости изменять код.

Затем мы реализуем несколько загрузчиков рыночных данных для гибкого формирования наборов данных. Функция load_csv() импортирует ранее экспортированные исторические данные OHLCV, проверяет наличие необходимых столбцов, нормализует временные метки к UTC, удаляет пропущенные значения и формирует стандартизированный DataFrame, готовый к конструированию признаков. Функция load_mt5_bars() подключается напрямую к MetaTrader 5 и получает исторические бары из терминала, что позволяет быстро переобучать модель без использования внешних CSV-файлов. Наконец, функция make_synthetic_bars() генерирует реалистичные рыночные данные на основе случайного блуждания для быстрого тестирования и отладки пайплайна, когда реальные рыночные данные недоступны. В совокупности эти загрузчики образуют единый слой загрузки данных, который обеспечивает стабильность, согласованность и адаптируемость среды обучения в различных сценариях разработки.

```
# ---------------------------------------------------------------------------
# Technical indicators  (no TA-Lib dependency)
# ---------------------------------------------------------------------------
def rsi(series: pd.Series, period: int = 14) -> pd.Series:
```

```
    delta = series.diff()
    gain  = delta.clip(lower=0).ewm(com=period-1, adjust=False).mean()
    loss  = (-delta.clip(upper=0)).ewm(com=period-1, adjust=False).mean()
    return (100 - 100 / (1 + gain / loss.replace(0, np.nan))).fillna(50)


def atr(df: pd.DataFrame, period: int = 14) -> pd.Series:
    tr = pd.concat([
        df["high"] - df["low"],
        (df["high"] - df["close"].shift(1)).abs(),
        (df["low"]  - df["close"].shift(1)).abs(),
    ], axis=1).max(axis=1)
    return tr.ewm(com=period-1, adjust=False).mean()


def adx(df: pd.DataFrame, period: int = 14) -> pd.Series:
    up, down = df["high"].diff(), -df["low"].diff()
    pdm = up.where((up > down) & (up > 0), 0.0)
    ndm = down.where((down > up) & (down > 0), 0.0)
    tr_s = atr(df, period)
    pdi  = 100 * pdm.ewm(com=period-1, adjust=False).mean() / tr_s.replace(0, np.nan)
    ndi  = 100 * ndm.ewm(com=period-1, adjust=False).mean() / tr_s.replace(0, np.nan)
    dx   = (100 * (pdi - ndi).abs() / (pdi + ndi).replace(0, np.nan)).fillna(0)
    return dx.ewm(com=period-1, adjust=False).mean().fillna(20)


# ---------------------------------------------------------------------------
# SMC signal detection  (mirrors EA logic exactly)
# ---------------------------------------------------------------------------
SIGNAL_OB  = 0
SIGNAL_FVG = 1
SIGNAL_BOS = 2


def _is_swing_high(high: np.ndarray, idx: int, length: int) -> bool:
    n = len(high)
    for i in range(1, length + 1):
        l, r = idx + i, idx - i
        if r < 0:                                   return False
        if high[idx] <= high[r]:                    return False
        if l < n and high[idx] < high[l]:           return False
    return True


def _is_swing_low(low: np.ndarray, idx: int, length: int) -> bool:
    n = len(low)
    for i in range(1, length + 1):
        l, r = idx + i, idx - i
        if r < 0:                                   return False
        if low[idx] >= low[r]:                      return False
        if l < n and low[idx] > low[l]:             return False
    return True


def detect_signals(df: pd.DataFrame, args) -> pd.DataFrame:
    point   = get_point(args.symbol)
    sw_len  = args.swing_len
```

```
    fvg_min = args.fvg_min * point

    H = df["high"].values
    L = df["low"].values
    O = df["open"].values
    C = df["close"].values
    n = len(df)
    events = []

    # ── Order Blocks ──────────────────────────────────────────────────────
    for i in range(4, n - 4):
        # Bullish OB: bearish candle at [i+3] before bullish impulse
        if (O[i+3] > C[i+3] and O[i+2] < C[i+2] and
                O[i] < C[i] and O[i+3] < C[i+2]):
            events.append(dict(bar_idx=i, signal_type=SIGNAL_OB, direction=1,
                               zone_high=H[i+3], zone_low=L[i+3],
                               entry_price=(H[i+3]+L[i+3])/2))
        # Bearish OB: bullish candle at [i+3] before bearish impulse
        if (O[i+3] < C[i+3] and O[i+2] > C[i+2] and
                O[i] > C[i] and O[i+3] > C[i+2]):
            events.append(dict(bar_idx=i, signal_type=SIGNAL_OB, direction=-1,
                               zone_high=H[i+3], zone_low=L[i+3],
                               entry_price=(H[i+3]+L[i+3])/2))

    # ── Fair Value Gaps ───────────────────────────────────────────────────
    for i in range(2, n - 2):
        la, ha = L[i+2], H[i+2]
        hc, lc = H[i],   L[i]
        if la > hc and (la - hc) >= fvg_min:
            events.append(dict(bar_idx=i, signal_type=SIGNAL_FVG, direction=1,
                               zone_high=la, zone_low=hc,
                               entry_price=(la+hc)/2))
        elif ha < lc and (lc - ha) >= fvg_min:
            events.append(dict(bar_idx=i, signal_type=SIGNAL_FVG, direction=-1,
                               zone_high=lc, zone_low=ha,
                               entry_price=(lc+ha)/2))

    # ── Break of Structure ────────────────────────────────────────────────
    sh_list = [i for i in range(sw_len, n-sw_len) if _is_swing_high(H, i, sw_len)]
    sl_list = [i for i in range(sw_len, n-sw_len) if _is_swing_low(L,  i, sw_len)]

    for sl_idx in sl_list:      # BOS Buy: break below swing low
        for j in range(sl_idx+1, min(sl_idx+50, n)):
            if L[j] < L[sl_idx]:
                events.append(dict(bar_idx=j, signal_type=SIGNAL_BOS, direction=1,
                                   zone_high=L[sl_idx]+fvg_min,
                                   zone_low =L[sl_idx]-fvg_min,
                                   entry_price=L[sl_idx]))
                break

    for sh_idx in sh_list:      # BOS Sell: break above swing high
        for j in range(sh_idx+1, min(sh_idx+50, n)):
            if H[j] > H[sh_idx]:
                events.append(dict(bar_idx=j, signal_type=SIGNAL_BOS, direction=-1,
                                   zone_high=H[sh_idx]+fvg_min,
                                   zone_low =H[sh_idx]-fvg_min,
                                   entry_price=H[sh_idx]))
```

```
                break

    sig_df = (pd.DataFrame(events)
                .drop_duplicates(subset=["bar_idx","signal_type","direction"])
                .sort_values("bar_idx")
                .reset_index(drop=True))

    print(f"[SMC]   Detected {len(sig_df):,} raw signal events  "
          f"(OB={(sig_df.signal_type==SIGNAL_OB).sum()}, "
          f"FVG={(sig_df.signal_type==SIGNAL_FVG).sum()}, "
          f"BOS={(sig_df.signal_type==SIGNAL_BOS).sum()})")
    return sig_df


# ---------------------------------------------------------------------------
# Labeller: walk-forward TP/SL simulation
# ---------------------------------------------------------------------------
def label_events(df: pd.DataFrame, sig_df: pd.DataFrame, args) -> pd.DataFrame:
    """
    Simulate each detected signal forward bar-by-bar.
    Label = 1 if TP is hit first, 0 if SL is hit first.
    Events where neither is hit within 2000 bars are dropped (expired).

    SL / TP are in EA 'points' - converted to price distance via get_point().
    """
    point    = get_point(args.symbol)
    sl_dist  = args.sl_pts * point
    tp_dist  = args.tp_pts * point
    H        = df["high"].values
    L        = df["low"].values
    n_bars   = len(df)
    LOOKFWD  = 2000

    print(f"[Label] Symbol: {args.symbol}  Point: {point}")
    print(f"[Label] SL: {args.sl_pts} pts = {sl_dist:.4f}  |  "
          f"TP: {args.tp_pts} pts = {tp_dist:.4f}  |  Lookahead: {LOOKFWD} bars")

    # Auto-scale safety net: if SL > 10× mean bar range, trades will rarely resolve
    mean_hl = float((df["high"] - df["low"]).mean())
    if sl_dist > 10 * mean_hl:
        sl_dist = round(2.0 * mean_hl, 5)
        tp_dist = round(4.0 * mean_hl, 5)
        print(f"[Label] WARN: SL/TP too large for data volatility.")
        print(f"[Label] Auto-scaled -> SL={sl_dist:.4f}  TP={tp_dist:.4f} (2x/4x mean H
        print(f"[Label] NOTE: This only applies to synthetic data. "
              "Real CSV data will use your exact sl_pts/tp_pts.")

    labels = []
    for _, row in sig_df.iterrows():
        entry    = float(row["entry_price"])
        start    = int(row["bar_idx"])
        d        = int(row["direction"])
        sl_price = entry - d * sl_dist
        tp_price = entry + d * tp_dist
        outcome  = np.nan

        for j in range(start + 1, min(start + LOOKFWD, n_bars)):
```

```
            if d == 1:    # buy
                if L[j] <= sl_price: outcome = 0; break
                if H[j] >= tp_price: outcome = 1; break
            else:         # sell
                if H[j] >= sl_price: outcome = 0; break
                if L[j] <= tp_price: outcome = 1; break

        labels.append(outcome)

    result = sig_df.copy()
    result["label"] = labels
    result.dropna(subset=["label"], inplace=True)
    result["label"] = result["label"].astype(int)

    expired  = len(sig_df) - len(result)
    win_rate = result["label"].mean()
    rr       = args.tp_pts / args.sl_pts

    print(f"\n[Label] Results:")
    print(f"        Total signals  : {len(sig_df):,}")
    print(f"        Labelled       : {len(result):,}  ({100*len(result)/len(sig_df):.1f
    print(f"        Expired (NaN)  : {expired:,}")
    print(f"        Win rate       : {win_rate:.1%}")
    print(f"        TP hits        : {result['label'].sum():,}")
    print(f"        SL hits        : {(result['label']==0).sum():,}")
    print(f"        Risk:Reward    : 1:{rr:.2f}")
    print(f"        Expected win%  : {100/(1+rr):.1f}%  (random baseline)\n")
    return result
```

Мы реализуем полнофункциональный уровень анализа рынка, который сначала рассчитывает основные технические индикаторы — RSI, ATR и ADX — с использованием чистых формул NumPy/Pandas без внешних зависимостей. Эти индикаторы помогают количественно оценить импульс, волатильность и силу тренда в соответствии с тем, как торговая система интерпретирует структуру рынка. Кроме того, мы создаём сигнальный модуль SMC, который повторяет логику советника, обнаруживая события ордер-блоков (OB), гэпов справедливой стоимости (FVG) и слома структуры (BOS) путём сканирования исторического ценового движения и определения свинговых максимумов и минимумов на основе настраиваемой длины структуры.

Затем с помощью модуля разметки с просмотром вперёд мы моделируем, как каждый обнаруженный сигнал отработал бы в реальных рыночных условиях. Для каждого события мы просматриваем движение цены бар за баром, чтобы определить, что будет достигнуто раньше — тейк-профит или стоп-лосс, используя согласованные с советником преобразования расстояний в пунктах. Сигналы, исход по которым не определяется в пределах фиксированного окна просмотра вперёд, отбрасываются как истёкшие, что гарантирует сохранение только значимых исходов. В итоге мы формируем чистый размеченный набор данных вместе с диагностической статистикой — долей прибыльных сделок, долей разрешённых исходов и соотношением риска к прибыли, — что создаёт основу для обучения модели машинного обучения.

```
# ---------------------------------------------------------------------------
# Feature engineering  (12 features — must match MQL5 RunONNX exactly)
# ---------------------------------------------------------------------------
FEATURE_COLS = [
    "f_signal_ob",          # one-hot: is OB signal?
    "f_signal_fvg",         # one-hot: is FVG signal?
    "f_signal_bos",         # one-hot: is BOS signal?
    "f_direction",          # +1 bullish / -1 bearish
```

```
    "f_zone_width_atr",      # zone width normalised by ATR(14)
    "f_dist_to_zone_atr",    # |entry - zone midpoint| / ATR(14)
    "f_fib_pct",             # retrace depth inside zone [0,1]
    "f_session_sin",         # hour-of-day sin encoding
    "f_session_cos",         # hour-of-day cos encoding
    "f_rsi14",               # RSI(14) / 100
    "f_adx14",               # ADX(14) / 100
    "f_spread_norm",         # H-L range / ATR(14)  (spread proxy)
]


def build_features(df: pd.DataFrame, sig_df: pd.DataFrame) -> pd.DataFrame:
    atr14 = atr(df, 14)
    rsi14 = rsi(df["close"], 14)
    adx14 = adx(df, 14)
    hours = df.index.hour

    rows = []
    for _, row in sig_df.iterrows():
        i     = max(0, min(int(row["bar_idx"]), len(df)-1))
        atr_v = max(float(atr14.iloc[i]), 1e-8)
        zw    = row["zone_high"] - row["zone_low"]
        zmid  = (row["zone_high"] + row["zone_low"]) / 2
        hour  = int(hours[i])

        rows.append({
            "f_signal_ob":        int(row["signal_type"] == SIGNAL_OB),
            "f_signal_fvg":       int(row["signal_type"] == SIGNAL_FVG),
            "f_signal_bos":       int(row["signal_type"] == SIGNAL_BOS),
            "f_direction":        float(row["direction"]),
            "f_zone_width_atr":   float(np.clip(zw / atr_v, 0, 10)),
            "f_dist_to_zone_atr": float(np.clip(abs(row["entry_price"] - zmid) / atr_v,
            "f_fib_pct":          float(np.clip(abs(row["entry_price"] - row["zone_low"
            "f_session_sin":      float(np.sin(2 * np.pi * hour / 24)),
            "f_session_cos":      float(np.cos(2 * np.pi * hour / 24)),
            "f_rsi14":            float(rsi14.iloc[i]) / 100.0,
            "f_adx14":            float(np.clip(adx14.iloc[i], 0, 100)) / 100.0,
            "f_spread_norm":      float(np.clip((df["high"].iloc[i] - df["low"].iloc[i]
        })

    feat_df = pd.DataFrame(rows, columns=FEATURE_COLS, index=sig_df.index)
    feat_df["label"] = sig_df["label"].values
    feat_df.dropna(inplace=True)
    print(f"[Feat]  Feature matrix: {feat_df.shape[0]:,} rows x {len(FEATURE_COLS)} fea
    return feat_df


# ---------------------------------------------------------------------------
# Training
# ---------------------------------------------------------------------------
def train(feat_df: pd.DataFrame, args):
    from sklearn.model_selection import StratifiedKFold, cross_val_score
    from sklearn.pipeline import Pipeline
    from sklearn.preprocessing import StandardScaler
    from sklearn.metrics import classification_report, roc_auc_score
    from xgboost import XGBClassifier
```

```
X = feat_df[FEATURE_COLS].values.astype(np.float32)
y = feat_df["label"].values.astype(int)

pos = int(y.sum())
neg = int(len(y) - pos)
w   = neg / max(pos, 1)
print(f"[Train] TP hits: {pos:,}  SL hits: {neg:,}  "
      f"scale_pos_weight: {w:.2f}")

if len(np.unique(y)) < 2:
    print("[Train] WARNING: only one outcome class - model will be trivial.")
    print("[Train] Fitting anyway for ONNX export. Re-check sl_pts / tp_pts.")
    pipe = Pipeline([("scaler", StandardScaler()),
                     ("xgb", XGBClassifier(n_estimators=100, random_state=42,
                                           use_label_encoder=False,
                                           eval_metric="logloss"))])
    pipe.fit(X, y)
    return pipe

xgb = XGBClassifier(
    n_estimators    = 400,
    max_depth       = 5,
    learning_rate   = 0.04,
    subsample       = 0.8,
    colsample_bytree = 0.8,
    min_child_weight = 3,
    gamma           = 0.1,
    scale_pos_weight = w,
    eval_metric     = "logloss",
    use_label_encoder = False,
    random_state    = 42,
    n_jobs          = -1,
)
pipeline = Pipeline([("scaler", StandardScaler()), ("xgb", xgb)])

n_splits = min(5, pos, neg)
cv = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=42)
auc = cross_val_score(pipeline, X, y, cv=cv, scoring="roc_auc", n_jobs=-1)
print(f"\n[Train] {n_splits}-Fold CV ROC-AUC: {auc.mean():.4f} +/- {auc.std():.4f}"

# Interpretation guide printed alongside results
if   auc.mean() >= 0.70: auc_comment = "Good - model adds real edge"
elif auc.mean() >= 0.62: auc_comment = "Fair - some predictive signal"
elif auc.mean() >= 0.55: auc_comment = "Weak - borderline useful"
else:                    auc_comment = "Poor - near random; check data quality"
print(f"        Interpretation  : {auc_comment}")

pipeline.fit(X, y)
y_pred = pipeline.predict(X)
y_prob = pipeline.predict_proba(X)[:, 1]

labels_present = sorted(np.unique(y))
tgt = ["SL/Expire","TP Hit"] if len(labels_present)==2 else [str(l) for l in labels
print("\n[Train] In-sample classification report:")
print(classification_report(y, y_pred, target_names=tgt, labels=labels_present))
print(f"[Train] In-sample ROC-AUC : {roc_auc_score(y, y_prob):.4f}  "
      "(will be higher than CV - expected due to overfitting)")
```

```
    imp = (pd.DataFrame({"feature": FEATURE_COLS,
                         "importance": xgb.feature_importances_})
           .sort_values("importance", ascending=False))
    print("\n[Train] Feature importances (higher = more useful to the model):")
    print(imp.to_string(index=False))

    return pipeline
```

Мы создаём структурированный слой конструирования признаков, который преобразует исходные сигналы SMC в фиксированное 12-мерное числовое представление. Данный формат полностью соответствует структуре ONNX-инференса в MQL5. Каждый сигнал кодируется с использованием сочетания структурной и контекстуальной информации. Сюда входят тип сигнала (ордер-блок (OB), гэп справедливой стоимости (FVG), слом структуры (BOS)), направление, геометрия зоны, нормированная по ATR, положение относительно коррекции Фибоначчи, а также метрики расстояния до зоны.

Мы также включаем признаки более широкого рыночного контекста, чтобы модель лучше учитывала рыночную ситуацию. Время торговой сессии кодируется с помощью синусного и косинусного преобразований. Значения RSI и ADX нормализуются, а волатильность учитывается через отношение спреда к ATR. Затем мы обучаем классификатор XGBoost прогнозировать, приведёт ли каждый сигнал к тейк-профиту (TP) или стоп-лоссу (SL). Этот процесс включает балансировку классов, перекрёстную валидацию и оценку по метрике ROC-AUC. В итоге мы получаем обученный пайплайн с понятной оценкой значимости признаков и готовностью к экспорту для развёртывания через ONNX в советнике MQL5.

```
# ---------------------------------------------------------------------------
# ONNX export + verification
# ---------------------------------------------------------------------------
def export_onnx(pipeline, out_dir: str, mt5_files_path: str = '') -> str:
    from skl2onnx import convert_sklearn, update_registered_converter
    from skl2onnx.common.data_types import FloatTensorType
    from skl2onnx.common.shape_calculator import calculate_linear_classifier_output_sha
    import onnxruntime as rt

    try:
        from xgboost import XGBClassifier as _XGB
        from onnxmltools.convert.xgboost.operator_converters.XGBoost import convert_xgb
        update_registered_converter(
            _XGB, "XGBoostXGBClassifier",
            calculate_linear_classifier_output_shapes,
            convert_xgboost,
            options={"nocl": [True, False], "zipmap": [True, False, "columns"]},
        )
    except ImportError:
        sys.exit("[ERROR] onnxmltools required.\n  pip install onnxmltools")

    n_feat = len(FEATURE_COLS)
    onnx_model = convert_sklearn(
        pipeline,
        initial_types=[("float_input", FloatTensorType([None, n_feat]))],
        options={"zipmap": False},
        target_opset={"": 15, "ai.onnx.ml": 3},
    )

    model_bytes = onnx_model.SerializeToString()

    # Always save a local copy
    out_path = os.path.join(out_dir, "smc_filter.onnx")
```

```
    with open(out_path, "wb") as f:
        f.write(model_bytes)
    print(f"\n[ONNX] Local copy saved  ->  {out_path}")
    print(f"       File size         :  {Path(out_path).stat().st_size / 1024:.1f} KB")

    # Save directly into MT5 Files folder
    if mt5_files_path and mt5_files_path.strip():
        mt5_dir = Path(mt5_files_path.strip())
        if mt5_dir.exists():
            mt5_dest = mt5_dir / "smc_filter.onnx"
            with open(mt5_dest, "wb") as f:
                f.write(model_bytes)
            print(f"[ONNX] MT5 copy saved    ->  {mt5_dest}")
        else:
            print(f"[ONNX] WARNING: mt5_files_path not found: {mt5_dir}")
            print(f"       Copy \"{out_path}\" to MQL5/Files/ manually.")
    else:
        print("[ONNX] mt5_files_path not set -- copy to MQL5/Files/ manually.")

    # Verify inference works
    sess     = rt.InferenceSession(out_path, providers=["CPUExecutionProvider"])
    dummy    = np.random.rand(1, n_feat).astype(np.float32)
    in_name  = sess.get_inputs()[0].name
    lbl_name = sess.get_outputs()[0].name
    prb_name = sess.get_outputs()[1].name
    lbl, prb = sess.run([lbl_name, prb_name], {in_name: dummy})
    print(f"[ONNX] Verification  ->  label: {lbl}  prob: {prb}")
    print(f"[ONNX] Input shape   :  {sess.get_inputs()[0].shape}")
    print(f"[ONNX] Output names  :  {[o.name for o in sess.get_outputs()]}")

    mt5_path = r"%APPDATA%\MetaQuotes\Terminal\<ID>\MQL5\Files\smc_filter.onnx"
    print(f"\n{'='*55}")
    print(f"  Next steps:")
    print(f"  1. Copy '{out_path}'")
    print(f"     -> {mt5_path}")
    print(f"     (Open MT5: File -> Open Data Folder to find <ID>)")
    print(f"  2. Compile SMC_AI_Filter.mq5 in MetaEditor")
    print(f"  3. Attach EA to XAUUSD H1 chart")
    print(f"  4. Run Strategy Tester on the same date range as your CSV")
    print(f"{'='*55}\n")
    return out_path

# ---------------------------------------------------------------------------
# Audit CSV
# ---------------------------------------------------------------------------
def save_report(feat_df: pd.DataFrame, out_dir: str):
    path = os.path.join(out_dir, "smc_features.csv")
    feat_df.to_csv(path, index=False)
    print(f"[Report] Labelled feature matrix saved -> {path}")
    print(f"         ({len(feat_df):,} rows - open in Excel to inspect signal quality)")

# ---------------------------------------------------------------------------
# Main
# ---------------------------------------------------------------------------
def main():
    cfg = get_config()
    Path(cfg.out_dir).mkdir(parents=True, exist_ok=True)
```

```
    print("=" * 55)
    print("  SMC AI Filter -- XGBoost Training Pipeline")
    print(f"  Data source : {cfg.data_source.upper()}")
    print(f"  Symbol      : {cfg.symbol}  |  TF: {cfg.timeframe}")
    print(f"  Point size  : {get_point(cfg.symbol)}")
    print(f"  SL: {cfg.sl_pts} pts  |  TP: {cfg.tp_pts} pts")
    print("=" * 55 + "\n")

    # 1. Load data
    src = cfg.data_source
    if src == "csv":
        df = load_csv(cfg.csv_file)
    elif src == "mt5":
        df = load_mt5_bars(cfg.symbol, cfg.timeframe, cfg.bars)
    elif src == "synthetic":
        df = make_synthetic_bars(
            getattr(cfg, "bars", 20000), cfg.symbol, cfg.timeframe)
    else:
        sys.exit(f"[ERROR] Unknown data_source '{src}'. Choose: csv | mt5 | synthetic")

    if len(df) < 500:
        sys.exit(f"[ERROR] Only {len(df)} bars loaded - need at least 500.")

    # 2. Detect SMC signals
    sig_df = detect_signals(df, cfg)
    if len(sig_df) == 0:
        sys.exit("[ERROR] No signals detected. Check symbol/data settings.")

    # 3. Label TP / SL outcomes
    sig_df = label_events(df, sig_df, cfg)
    if len(sig_df) < 100:
        sys.exit(
            f"[ERROR] Only {len(sig_df)} labelled events (need >= 100).\n"
            "  - Extend the date range in fetch_historical_data.py\n"
            "  - Check that sl_pts / tp_pts match the actual price scale"
        )

    # 4. Feature engineering
    feat_df = build_features(df, sig_df)

    # 5. Train
    pipeline = train(feat_df, cfg)

    # 6. Export ONNX
    export_onnx(pipeline, cfg.out_dir, getattr(cfg, 'mt5_files_path', ''))

    # 7. Audit report
    save_report(feat_df, cfg.out_dir)

    print("Pipeline complete.")


if __name__ == "__main__":
    main()
```

Здесь мы реализуем этап экспорта в ONNX и проверки, который преобразует обученный пайплайн XGBoost в формат, совместимый с MetaTrader 5. Сначала мы регистрируем конвертер XGBoost и преобразуем пайплайн Scikit-learn в граф ONNX. Размерность входного тензора зафиксирована так, чтобы соответствовать 12 признакам, полученным при конструировании. Затем модель сериализуется и сохраняется локально. Её также можно скопировать непосредственно в каталог MQL5/Files платформы MetaTrader 5 для немедленного использования советником.

После экспорта мы проверяем модель с помощью ONNX Runtime. Через модель пропускается фиктивный входной вектор, чтобы проверить корректность размерности входа, выходных меток и вероятностных оценок. Это гарантирует полную работоспособность модели перед развертыванием. Затем мы определяем основной пайплайн, который управляет всем рабочим процессом. Мы загружаем набор данных, обнаруживаем SMC-сигналы и размечаем каждое событие с помощью симуляции TP и SL. Далее мы преобразуем сигналы в векторы признаков и обучаем модель XGBoost. В заключение мы экспортируем модель в формат ONNX и формируем отчёт аудита, завершая создание полноценной сквозной торговой системы на базе искусственного интеллекта, готовой к интеграции с MetaTrader 5.

Результат:

```
[Config] Jupyter detected -- using NOTEBOOK_CONFIG.
[Config] Edit the NOTEBOOK_CONFIG dict at the top of this file.

============================================================
  SMC AI Filter -- XGBoost Training Pipeline
  Data source : CSV
  Symbol      : XAUUSD.m  |  TF: H1
  Point size  : 0.01
  SL: 3500 pts  |  TP: 7500 pts
============================================================

[CSV]   Loaded 13,584 bars from 'XAUUSD_H1.csv'
        Date range : 2023-09-04  ->  2025-12-31
        Price range: 1815.03  ->  4546.69
        Mean H-L   : 8.1656  (8.17 for XAUUSD-style)

[SMC]   Detected 5,019 raw signal events  (OB=1312, FVG=2761, BOS=946)
[Label] Symbol: XAUUSD.m  Point: 0.01
[Label] SL: 3500 pts = 35.0000  |  TP: 7500 pts = 75.0000  |  Lookahead: 2000 bars

[Label] Results:
        Total signals  : 5,019
        Labelled       : 5,014  (99.9% resolved)
        Expired (NaN)  : 5
        Win rate       : 36.8%
        TP hits        : 1,846
        SL hits        : 3,168
        Risk:Reward    : 1:2.14
        Expected win%  : 31.8%  (random baseline)

[Feat]  Feature matrix: 5,014 rows x 12 features
[Train] TP hits: 1,846  SL hits: 3,168  scale_pos_weight: 1.72

[Train] 5-Fold CV ROC-AUC: 0.7118 +/- 0.0178
        Interpretation  : Good — model adds real edge
```

```
[Train] In-sample classification report:
              precision    recall  f1-score   support

   SL/Expire       0.90      0.76      0.82      3168
      TP Hit       0.67      0.86      0.75      1846

    accuracy                           0.79      5014
   macro avg       0.79      0.81      0.79      5014
weighted avg       0.82      0.79      0.80      5014

[Train] In-sample ROC-AUC : 0.9124  (will be higher than CV — expected due to overfitting)

[Train] Feature importances (higher = more useful to the model):
           feature  importance
       f_direction    0.531883
      f_signal_bos    0.084700
      f_signal_fvg    0.060759
       f_signal_ob    0.051301
  f_zone_width_atr    0.048476
          f_adx14    0.046625
          f_rsi14    0.045798
     f_spread_norm    0.045515
     f_session_sin    0.043334
     f_session_cos    0.041610
         f_fib_pct    0.000000
f_dist_to_zone_atr    0.000000

[ONNX] Local copy saved  ->  .\smc_filter.onnx
       File size         :  539.1 KB
```

Интеграция с MQL5

```
//+------------------------------------------------------------------+
//|                                                       SMC AI.mq5 |
//|           SMC EA with XGBoost ONNX inference, zone visualisation |
//|                                  Copyright 2025, MetaQuotes Ltd. |
//|                     https://www.mql5.com/en/users/johnhlomohang/ |
//+------------------------------------------------------------------+
#property copyright "Copyright 2025, MetaQuotes Ltd."
#property link      "https://www.mql5.com/en/users/johnhlomohang/"
#property version   "1.00"

#resource "\\Files\\SMConnx\\smc_filter.onnx" as uchar ExtModel[]

#include <Trade/Trade.mqh>
#include <Trade/PositionInfo.mqh>

CTrade         trade;
CPositionInfo  pos;

//+------------------------------------------------------------------+
//|  ENUMS                                                           |
//+------------------------------------------------------------------+
enum ENUM_STRATEGY  { STRAT_OB, STRAT_FVG, STRAT_BOS, STRAT_AUTO };
enum ENUM_SWING_TYPE { SWING_OB, SWING_BOS };
enum ENUM_TREND_STR { TS_WEAK, TS_MEDIUM, TS_STRONG };

//+------------------------------------------------------------------+
```

```
//|  INPUTS                                                          |
//+------------------------------------------------------------------+
input group              "═══  Strategy  ═══"
input ENUM_STRATEGY      TradeStrategy   = STRAT_AUTO;
input double             In_Lot          = 0.02;
input long               MagicNumber     = 76543;

input group              "═══  SL / TP  ═══"
enum ENUM_SLTP_MODE { PRICE_DIST, ATR_MULT };
input ENUM_SLTP_MODE     SL_Mode         = PRICE_DIST;
input double             StopLoss_Dist   = 20.0;    // SL in price units ($)
input double             TakeProfit_Dist = 50.0;    // TP in price units ($)

//--- ATR multiplier (used when SL_Mode = ATR_MULT) ---
input double             SL_ATR_Mult     = 1.5;     // SL = ATR(14) × this
input double             TP_ATR_Mult     = 3.0;     // TP = ATR(14) × this

//--- Trailing stop ---
input bool               Trail_SL           = true;
input double             Trail_TriggerDist  = 10.0;  // profit ($) needed to start trai
input double             Trail_StepDist     = 5.0;   // trail step ($) - SL moves in th

input group              "═══  SMC Parameters  ═══"
input int                SwingPeriod     = 5;        // bars each side for swing
input double             Fib_Trade_lvls  = 61.8;     // OB fib retrace %
input bool               DrawBOSLines    = true;
input int                FVG_MinPoints   = 3;
input int                FVG_ScanBars    = 20;
input bool               FVG_TradeAtEQ   = true;
input bool               OneTradePerBar  = true;

input group              "═══  ONNX AI Filter  ═══"
input bool               UseAI           = true;
input double             AI_MinScore     = 0.60;    // min probability to allow trade
input double             AI_StrongScore  = 0.80;    // threshold for "Strong" trend rat
input double             AI_MediumScore  = 0.65;    // threshold for "Medium" trend rat

input group              "═══  Panel  ═══"
input int                PanelX          = 20;
input int                PanelY          = 30;
input color              PanelBG         = C'30,30,40';
input color              PanelText       = clrSilver;
input color              PanelAccent     = clrDodgerBlue;

//+------------------------------------------------------------------+
//| CONSTANTS                                                        |
//+------------------------------------------------------------------+
#define CLR_BULL_OB     clrLimeGreen
#define CLR_BEAR_OB     clrTomato
#define CLR_BULL_FVG    clrDodgerBlue
#define CLR_BEAR_FVG    clrOrange
#define CLR_BOS_BULL    clrDodgerBlue
#define CLR_BOS_BEAR    clrTomato
#define N_FEATURES      12

//+------------------------------------------------------------------+
//| GLOBALS                                                          |
```

```
//+------------------------------------------------------------------+
double   Bid, Ask;
datetime g_lastBarTime = 0;

//--- ONNX
long     g_onnx_model  = INVALID_HANDLE;
float    g_ai_score    = 0.0f;

//--- Panel state
string   g_panel_signal = "-";
string   g_panel_origin = "-";
ENUM_TREND_STR g_trend_str = TS_WEAK;

//--- OB state
struct SOrderBlock
  {
   int               direction;
   datetime          time;
   double            high, low;
  };
SOrderBlock  g_OB;
bool         g_OB_valid       = false;
datetime     g_lastOBTradeTime = 0;

//--- OB fib scratch
double   fib_high, fib_low;
datetime fib_t1, fib_t2;

//--- BOS state
double   swng_High = -1.0, swng_Low = -1.0;
datetime bos_tH = 0, bos_tL = 0;
bool     Bull_BOS_traded = false, Bear_BOS_traded = false;
datetime lastBOSTradeTime = 0;
int      lastBOSTradeDirection = 0;

//--- FVG trade guard
datetime lastFVGTradeBar = 0;
```

Для начала определим перечисления и входные параметры, которые управляют поведением советника. Перечисление ENUM_STRATEGY позволяет трейдеру выбирать между запуском ордер-блока (OB), гэпа справедливой стоимости (FVG), слома структуры (BOS) или всех трёх сразу с помощью опции STRAT_AUTO. Перечисление ENUM_SLTP_MODE позволяет выбрать между фиксированными, рассчитываемыми в долларах уровнями стоп-лосса и тейк-профита и уровнями, которые автоматически масштабируются по ATR. Здесь мы также объявляем все глобальные переменные, включая дескриптор модели ONNX, состояние отображения панели и отдельные трекеры для каждой концепции SMC.

Директива #resource, расположенная в самом начале кода, отвечает за встраивание обученной модели ONNX в скомпилированный советник. После компиляции модель находится внутри файла .ex5 и не требует внешних файлов во время выполнения. Именно это обеспечивает переносимость советника между компьютерами и его совместимость с Тестером стратегий без каких-либо ошибок, связанных с путями к файлам.

```
//+------------------------------------------------------------------+
//| Expert initialization function                                   |
//+------------------------------------------------------------------+
int OnInit()
  {
```

```
   if(!InitONNX())
      return INIT_FAILED;
   trade.SetExpertMagicNumber(MagicNumber);
   CreatePanel();
   UpdatePanel();
   return INIT_SUCCEEDED;
  }

//+------------------------------------------------------------------+
//| Expert deinitialization function                                 |
//+------------------------------------------------------------------+
void OnDeinit(const int reason)
  {
   DeinitONNX();
   DestroyPanel();
  }

//+------------------------------------------------------------------+
//| Expert tick function                                             |
//+------------------------------------------------------------------+
void OnTick()
  {
   //--- Trail SL runs on every tick so it reacts immediately to price movement.
   ManageTrailingSL();

   //--- Signal detection and trade entry only on new bar open.
   if(!IsNewBar())
      return;

   if(TradeStrategy == STRAT_FVG || TradeStrategy == STRAT_AUTO)
      DetectAndDrawFVGs();

   if(TradeStrategy == STRAT_OB || TradeStrategy == STRAT_AUTO)
      DetectAndDrawOrderBlocks();

   if(TradeStrategy == STRAT_BOS || TradeStrategy == STRAT_AUTO)
      DetectAndDrawBOS();
  }
```

Мы настраиваем советник в функции OnInit, загружая модель ONNX, регистрируя магический номер и отрисовывая торговую панель на графике. Если модель не загружается, мы сразу же возвращаем значение INIT_FAILED, чтобы советник не работал в некорректном состоянии. При удалении функция OnDeinit освобождает дескриптор модели и очищает все объекты панели. В OnTick мы сознательно разделяем две задачи. Трейлинг-стоп обновляется при каждом тике, поскольку должен реагировать на движение цены в режиме реального времени. Однако обнаружение сигнала и вход в сделку выполняются только при открытии нового бара. Это предотвращает открытие советником нескольких входов в пределах одной свечи и обеспечивает соответствие логики структуре сигналов SMC.

```
//+------------------------------------------------------------------+
//| HELPERS - price access                                           |
//+------------------------------------------------------------------+
double   getHigh(int i)     { return iHigh(_Symbol, _Period, i);  }
double   getLow(int i)      { return iLow(_Symbol, _Period, i);   }
double   getOpen(int i)     { return iOpen(_Symbol, _Period, i);  }
double   getClose(int i)    { return iClose(_Symbol, _Period, i); }
datetime getTime(int i)     { return iTime(_Symbol, _Period, i);  }
```

```
bool IsNewBar()
  {
   datetime t = (datetime)SeriesInfoInteger(_Symbol, _Period, SERIES_LASTBAR_DATE);
   if(g_lastBarTime == 0)
     {
      g_lastBarTime = t;
      return false;
     }
   if(g_lastBarTime != t)
     {
      g_lastBarTime = t;
      return true;
     }
   return false;
  }
```

Мы определяем набор небольших вспомогательных функций для доступа к значениям OHLC по индексу бара. Они служат обёртками над стандартными функциями рядов MQL5 и сохраняют читаемость остального кода. Вместо того чтобы повторять вызов iHigh(_Symbol, _Period, i) по всему советнику, мы вызываем функцию getHigh(i). Функция IsNewBar сравнивает время последнего известного бара с текущим. Если они различаются, функция обновляет сохранённое время и возвращает значение true. Это позволяет нам использовать простой и надёжный способ выполнения сигнальной логики только на открытии нового бара без использования счётчиков тиков или таймеров.

```
//+------------------------------------------------------------------+
//|  POSITION HELPERS                                                |
//+------------------------------------------------------------------+
bool HasOpenPosition()
  {
   for(int i = PositionsTotal() - 1; i >= 0; i--)
     {
      if(pos.SelectByIndex(i))
         if(pos.Symbol() == _Symbol && pos.Magic() == MagicNumber)
            return true;
     }
   return false;
  }

//+------------------------------------------------------------------+
//|  MANAGE TRAILING SL                                              |
//+------------------------------------------------------------------+
void ManageTrailingSL()
  {
   if(!Trail_SL)
      return;

   for(int i = PositionsTotal() - 1; i >= 0; i--)
     {
      if(!pos.SelectByIndex(i))
         continue;
      if(pos.Symbol() != _Symbol || pos.Magic() != MagicNumber)
         continue;

      double entry     = pos.PriceOpen();
```

```
      double current_sl = pos.StopLoss();
      double tp         = pos.TakeProfit();
      ulong  ticket     = pos.Ticket();

      Bid = SymbolInfoDouble(_Symbol, SYMBOL_BID);
      Ask = SymbolInfoDouble(_Symbol, SYMBOL_ASK);

      double new_sl = 0;

      if(pos.PositionType() == POSITION_TYPE_BUY)
        {
         double profit_dist = Bid - entry;
         if(profit_dist < Trail_TriggerDist)
            continue;  // not in profit enough yet

         //--- Trail: SL follows Bid minus one step
         new_sl = NormalizeDouble(Bid - Trail_StepDist, _Digits);

         //--- Only move SL if it improves (moves up)
         if(new_sl <= current_sl)
            continue;
        }
      else // SELL
        {
         double profit_dist = entry - Ask;
         if(profit_dist < Trail_TriggerDist)
            continue;

         //--- Trail: SL follows Ask plus one step
         new_sl = NormalizeDouble(Ask + Trail_StepDist, _Digits);

         //--- Only move SL if it improves (moves down)
         if(new_sl >= current_sl)
            continue;
        }

      //--- Validate new SL respects broker minimum stop distance
      double min_stop = (double)SymbolInfoInteger(_Symbol, SYMBOL_TRADE_STOPS_LEVEL) *
      double price_ref = (pos.PositionType() == POSITION_TYPE_BUY) ? Bid : Ask;
      if(MathAbs(price_ref - new_sl) < min_stop)
         continue;

      trade.PositionModify(ticket, new_sl, tp);
      Print("[Trail] SL moved to ", DoubleToString(new_sl, _Digits),
            "  (profit dist: ",
            DoubleToString((pos.PositionType()==POSITION_TYPE_BUY)
                           ? Bid - entry : entry - Ask, 2), ")");
     }
  }
```

Перед открытием новой позиции мы используем функцию HasOpenPosition, чтобы проверить, есть ли у советника открытая позиция по данному инструменту. Функция проходит по всем открытым позициям и сопоставляет их как по инструменту, так и по магическому номеру. Именно это обеспечивает соблюдение правила «одна сделка на один сигнал» во всех трёх стратегиях SMC.

Функция ManageTrailingSL запускается при каждом тике и управляет открытой позицией независимо от сигнальной логики. Как только расстояние, пройденное сделкой в прибыли, превышает Trail_TriggerDist,

стоп-лосс начинает следовать за ценой с шагом Trail_StepDist. SL перемещается только в сторону прибыли и никогда не расширяется. Кроме того, перед отправкой запроса на изменение мы проверяем новый SL на соответствие минимальному уровню стопов брокера.

```
//+------------------------------------------------------------------+
//| SL / TP CALCULATOR                                               |
//+------------------------------------------------------------------+
void CalcSLTP(double &sl_dist, double &tp_dist)
  {
   if(SL_Mode == ATR_MULT)
     {
      //--- ATR(14) - simple average of last 14 true ranges
      double atr = 0;
      for(int k = 1; k <= 14; k++)
        {
         double hl  = getHigh(k) - getLow(k);
         double hpc = MathAbs(getHigh(k) - getClose(k+1));
         double lpc = MathAbs(getLow(k)  - getClose(k+1));
         atr += MathMax(hl, MathMax(hpc, lpc));
        }
      atr /= 14.0;
      if(atr <= 0)
         atr = 10.0 * _Point;   // fallback

      sl_dist = atr * SL_ATR_Mult;
      tp_dist = atr * TP_ATR_Mult;
     }
   else // PRICE_DIST - direct dollar/price distance
     {
      sl_dist = StopLoss_Dist;
      tp_dist = TakeProfit_Dist;
     }

   //--- Safety floor: SL must be at least 5× the spread / minimum stop level
   double min_stop = (double)SymbolInfoInteger(_Symbol, SYMBOL_TRADE_STOPS_LEVEL) * _Po
   double spread   = SymbolInfoDouble(_Symbol, SYMBOL_ASK) - SymbolInfoDouble(_Symbol,
   double floor    = MathMax(min_stop, spread * 3.0);
   if(sl_dist < floor)
      sl_dist = floor;
   if(tp_dist < floor)
      tp_dist = floor;
  }
```

Мы централизуем расчёт расстояний/уровней SL и TP в функции CalcSLTP. Когда для параметра SL_Mode установлено значение PRICE_DIST, функция возвращает фиксированные значения в долларах, заданные во входных параметрах. При выборе ATR_MULT рассчитывается 14-периодный ATR и умножается на коэффициенты, заданные во входных параметрах. Затем оба результата проверяются на соответствие безопасному минимуму, рассчитанному на основе минимального уровня стопов брокера и текущего спреда. Наличие единой функции, отвечающей за это, означает, что любое изменение расчёта расстояний SL или TP автоматически одинаково применяется к сделкам по ордер-блокам (OB), гэпам справедливой стоимости (FVG) и сломам структуры (BOS). В самой функции выполнения ничего не задано жёстко.

```
//+------------------------------------------------------------------+
//|  ONNX INIT / DEINIT                                              |
//+------------------------------------------------------------------+
```

```
bool InitONNX()
  {
   if(!UseAI)
      return true;

   //--- Load from compiled resource (embedded at compile time via #resource)
   g_onnx_model = OnnxCreateFromBuffer(ExtModel, ONNX_DEFAULT);
   if(g_onnx_model == INVALID_HANDLE)
     {
      Print("[AI] ONNX load failed (", GetLastError(), "). "
            "Ensure smc_filter.onnx was in MQL5/Files/ when you compiled.");
      return false;
     }

   //--- Dynamic shapes: input [1 × N_FEATURES], output [1 × 2] (class label + proba)
   ulong in_shape[]  = {1, N_FEATURES};
   ulong out_shape[] = {1};          // predicted label
   ulong prb_shape[] = {1, 2};       // [prob_0, prob_1]

   if(!OnnxSetInputShape(g_onnx_model, 0, in_shape))
     { Print("[AI] OnnxSetInputShape failed"); return false; }

   if(!OnnxSetOutputShape(g_onnx_model, 0, out_shape))
     { Print("[AI] OnnxSetOutputShape[0] failed"); return false; }

   if(!OnnxSetOutputShape(g_onnx_model, 1, prb_shape))
     { Print("[AI] OnnxSetOutputShape[1] failed"); return false; }

   Print("[AI] ONNX model loaded from compiled resource. Features: ", N_FEATURES);
   return true;
  }

void DeinitONNX()
  {
   if(g_onnx_model != INVALID_HANDLE)
     { OnnxRelease(g_onnx_model); g_onnx_model = INVALID_HANDLE; }
  }

//+------------------------------------------------------------------+
//| ONNX INFERENCE                                                   |
//+------------------------------------------------------------------+
float RunONNX(int signal_type,  // SIGNAL_OB=0 FVG=1 BOS=2
              int direction,    // +1 bull / -1 bear
              double zone_high,
              double zone_low,
              double entry_price)
  {
   if(!UseAI || g_onnx_model == INVALID_HANDLE)
      return 1.0f; // pass-through

   double atr14 = 0.0;
     {
      //--- Compute ATR(14) as price distance (same units as SL/TP dollars)
      double sumTR = 0;
      for(int k = 1; k <= 14; k++)
        {
         double hl  = getHigh(k) - getLow(k);
```

```
      double hpc = MathAbs(getHigh(k) - getClose(k+1));
      double lpc = MathAbs(getLow(k)  - getClose(k+1));
      sumTR += MathMax(hl, MathMax(hpc, lpc));
     }
   atr14 = sumTR / 14.0;
  }
if(atr14 <= 0)
   atr14 = _Point;

double zone_mid   = (zone_high + zone_low) / 2.0;
double zone_width = zone_high - zone_low;
int    hour       = datetime(TimeCurrent());

//--- RSI(14) - Wilder EMA approximation
double rsi_val = 50.0;
  {
   double gain = 0, loss = 0;
   for(int k = 1; k <= 14; k++)
     {
      double d = getClose(k) - getClose(k+1);
      if(d > 0)
         gain += d;
      else
         loss -= d;
     }
   gain /= 14;
   loss /= 14;
   if(loss > 0)
      rsi_val = 100.0 - 100.0 / (1.0 + gain / loss);
  }

//--- ADX(14) approximation
double adx_val = 20.0;
  {
   double pdm = 0, ndm = 0, tr_sum = 0;
   for(int k = 1; k <= 14; k++)
     {
      double up   = getHigh(k) - getHigh(k+1);
      double down = getLow(k+1) - getLow(k);
      pdm   += (up > down && up > 0)   ? up   : 0;
      ndm   += (down > up && down > 0) ? down : 0;
      double hl  = getHigh(k) - getLow(k);
      double hpc = MathAbs(getHigh(k) - getClose(k+1));
      double lpc = MathAbs(getLow(k)  - getClose(k+1));
      tr_sum += MathMax(hl, MathMax(hpc, lpc));
     }
   if(tr_sum > 0)
     {
      double pdi = 100 * pdm / tr_sum;
      double ndi = 100 * ndm / tr_sum;
      if(pdi + ndi > 0)
         adx_val = 100 * MathAbs(pdi - ndi) / (pdi + ndi);
     }
  }

//--- Build float feature vector - order must match Python FEATURE_COLS
vectorf features;
```

```
   features.Init(N_FEATURES);
   features[0]  = (float)(signal_type == 0 ? 1 : 0);
   features[1]  = (float)(signal_type == 1 ? 1 : 0);
   features[2]  = (float)(signal_type == 2 ? 1 : 0);
   features[3]  = (float)direction;
   features[4]  = (float)MathMin(zone_width / atr14, 10.0);
   features[5]  = (float)MathMin(MathAbs(entry_price - zone_mid) / atr14, 10.0);
   features[6]  = (float)MathMin(MathAbs(entry_price - zone_low) / MathMax(zone_width,
   features[7]  = (float)MathSin(2.0 * M_PI * hour / 24.0);
   features[8]  = (float)MathCos(2.0 * M_PI * hour / 24.0);
   features[9]  = (float)(rsi_val / 100.0);
   features[10] = (float)MathMin(adx_val / 100.0, 1.0);
   features[11] = (float)MathMin((getHigh(0) - getLow(0)) / atr14, 5.0);

   //--- Run inference
   vectorf label_out(1), prob_out(2);
   if(!OnnxRun(g_onnx_model, ONNX_DEFAULT, features, label_out, prob_out))
     {
      Print("[AI] OnnxRun failed: ", GetLastError());
      return -1.0f;
     }

   return prob_out[1];  // probability of TP-hit
  }
```

Здесь мы инициализируем модель в InitONNX с помощью функции OnnxCreateFromBuffer, которая загружает данные непосредственно из скомпилированного ресурса, а не из файла по указанному пути. Затем мы явно задаём размерности входных и выходных тензоров, поскольку модель ожидает фиксированный вектор из 12 признаков типа float и возвращает прогнозируемую метку вместе с массивом вероятностей.

Функция RunONNX формирует вектор признаков в момент срабатывания сигнала. Она рассчитывает ATR, RSI и ADX на основе сырых ценовых данных, кодирует час торговой сессии с помощью синусного и косинусного преобразований, а также нормализует геометрию зоны относительно ATR. Функция возвращает значение prob_out[1], представляющее собой оцененную моделью вероятность того, что сделка достигнет уровня тейк-профита. Если оценка опускается ниже AI_MinScore, сделка блокируется.

```
//+------------------------------------------------------------------+
//| TRADE EXECUTION                                                  |
//+------------------------------------------------------------------+
bool ExecuteTrade(ENUM_ORDER_TYPE type, string origin)
  {
   double price = (type == ORDER_TYPE_BUY)
                  ? SymbolInfoDouble(_Symbol, SYMBOL_ASK)
                  : SymbolInfoDouble(_Symbol, SYMBOL_BID);

   //--- Get SL/TP as price distances (dollars for XAUUSD)
   double sl_dist, tp_dist;
   CalcSLTP(sl_dist, tp_dist);

   double sl = (type == ORDER_TYPE_BUY) ? price - sl_dist : price + sl_dist;
   double tp = (type == ORDER_TYPE_BUY) ? price + tp_dist : price - tp_dist;
   sl = NormalizeDouble(sl, _Digits);
   tp = NormalizeDouble(tp, _Digits);

   Print("[Trade] ", (type==ORDER_TYPE_BUY?"BUY":"SELL"),
         "  Entry:", DoubleToString(price,_Digits),
```

```
          "  SL:",    DoubleToString(sl,_Digits), " (", DoubleToString(sl_dist,2), ")",
          "  TP:",    DoubleToString(tp,_Digits), " (", DoubleToString(tp_dist,2), ")",
          "  Origin:", origin);

   trade.SetExpertMagicNumber(MagicNumber);
   bool ok = trade.PositionOpen(_Symbol, type, In_Lot, price, sl, tp, "SMC-AI");

   if(ok)
     {
      g_panel_signal = (type == ORDER_TYPE_BUY) ? "Buy" : "Sell";
      g_panel_origin = origin;
      g_trend_str    = ScoreToStrength(g_ai_score);
      UpdatePanel();
     }
   return ok;
  }
```

Функция ExecuteTrade отвечает за отправку всех ордеров. Она получает текущую рыночную цену, вызывает функцию CalcSLTP для расчёта расстояний, вычисляет ценовые уровни SL и TP, нормализует их с учётом точности котировки инструмента, а затем открывает позицию. Для каждой сделки в журнал выводится строка с указанием направления, цены входа, стоп-лосса (SL), тейк-профита (TP) и источника сигнала. После успешного открытия позиции мы обновляем панель, чтобы отобразить новый сигнал, источник и силу тренда. Метка силы тренда — «Сильный», «Средний» или «Слабый» — определяется непосредственно на основе оценки ИИ с помощью функции ScoreToStrength.

```
//+------------------------------------------------------------------+
//|  SWING DETECTION (unified)                                       |
//+------------------------------------------------------------------+
void DetectSwingForBar(int barIndex, ENUM_SWING_TYPE type)
  {
   const int len = SwingPeriod;
   bool isSwingH = true, isSwingL = true;

   int totalBars = Bars(_Symbol, _Period);

   for(int i = 1; i <= len; i++)
     {
      int right = barIndex - i;
      int left  = barIndex + i;
      if(right < 0)
        {
         isSwingH = false;
         isSwingL = false;
         break;
        }

      if(getHigh(barIndex) <= getHigh(right))
         isSwingH = false;
      if(left < totalBars && getHigh(barIndex) < getHigh(left))
         isSwingH = false;

      if(getLow(barIndex) >= getLow(right))
         isSwingL = false;
      if(left < totalBars && getLow(barIndex) > getLow(left))
         isSwingL = false;
     }
```

```
   if(type == SWING_OB)
     {
      if(isSwingH)
        {
         fib_high = getHigh(barIndex);
         fib_t1 = getTime(barIndex);
        }
      if(isSwingL)
        {
         fib_low  = getLow(barIndex);
         fib_t2 = getTime(barIndex);
        }
     }
   else
     {
      if(isSwingH)
        {
         swng_High = getHigh(barIndex);
         bos_tH = getTime(barIndex);
        }
      if(isSwingL)
        {
         swng_Low  = getLow(barIndex);
         bos_tL = getTime(barIndex);
        }
     }
  }
```

Мы используем одну унифицированную функцию DetectSwingForBar для определения свинговых максимумов и минимумов как в стратегии ордер-блоков (OB), так и в стратегии слома структуры (BOS). Она проверяет настраиваемое количество баров слева и справа от рассматриваемого бара. Если максимум бара является самым высоким в этом диапазоне, он считается свинговым максимумом. Та же логика применяется в обратном порядке для свинговых минимумов. Параметр SWING_TYPE определяет, где сохраняется результат. При вызове с параметром SWING_OB функция записывает данные в черновые переменные Фибоначчи, используемые стратегией ордер-блоков (OB). При вызове с параметром SWING_BOS она записывает данные в глобальные переменные свингов BOS. Это не позволяет двум стратегиям мешать друг другу при работе с опорными точками.

```
//+------------------------------------------------------------------+
//|  VISUALISATION HELPERS                                           |
//+------------------------------------------------------------------+
void DrawZoneRect(string name, datetime t1, datetime t2,
                  double high, double low, color clr, string label)
  {
   //--- Rectangle
   if(ObjectFind(0, name) == -1)
      ObjectCreate(0, name, OBJ_RECTANGLE, 0, t1, high, t2, low);
   ObjectSetInteger(0, name, OBJPROP_COLOR,    clr);
   ObjectSetInteger(0, name, OBJPROP_FILL,     true);
   ObjectSetInteger(0, name, OBJPROP_BACK,     true);
   ObjectSetInteger(0, name, OBJPROP_WIDTH,    1);
   ObjectSetInteger(0, name, OBJPROP_STYLE,    STYLE_SOLID);

   //--- Centred label – placed at the midpoint time (approximate) and price
   string lbl_name = name + "_lbl";
```

```
    datetime t_mid  = t1 + (datetime)((t2 - t1) / 2);
    double   p_mid  = (high + low) / 2.0;

    if(ObjectFind(0, lbl_name) == -1)
       ObjectCreate(0, lbl_name, OBJ_TEXT, 0, t_mid, p_mid);
    ObjectSetString(0,  lbl_name, OBJPROP_TEXT,      label);
    ObjectSetInteger(0, lbl_name, OBJPROP_COLOR,     clrWhite);
    ObjectSetInteger(0, lbl_name, OBJPROP_FONTSIZE,  9);
    ObjectSetInteger(0, lbl_name, OBJPROP_ANCHOR,    ANCHOR_CENTER);
   }

//--- Update the right-side time anchor of a zone rec
void ExtendZoneRect(string name)
   {
    if(ObjectFind(0, name) == -1)
       return;
    datetime tNow = (datetime)SeriesInfoInteger(_Symbol, _Period, SERIES_LASTBAR_DATE);
    ObjectSetInteger(0, name, OBJPROP_TIME, 1, tNow);

    string lbl_name = name + "_lbl";
    if(ObjectFind(0, lbl_name) == -1)
       return;
    //--- Recentre label horizontally
    datetime t1 = (datetime)ObjectGetInteger(0, name, OBJPROP_TIME, 0);
    datetime t_mid = t1 + (datetime)((tNow - t1) / 2);
    ObjectSetInteger(0, lbl_name, OBJPROP_TIME, 0, t_mid);
   }

void DeleteZone(string name)
   {
    ObjectDelete(0, name);
    ObjectDelete(0, name + "_lbl");
   }

//--- Draw BOS trend line with direction label
void DrawBOS(const string name, datetime t1, double p1,
             datetime t2, double p2, color col, int dir)
   {
    if(ObjectFind(0, name) != -1)
       return;
    ObjectCreate(0, name, OBJ_TREND, 0, t1, p1, t2, p2);
    ObjectSetInteger(0, name, OBJPROP_COLOR, col);
    ObjectSetInteger(0, name, OBJPROP_WIDTH, 2);
    ObjectSetInteger(0, name, OBJPROP_RAY_RIGHT, false);

    string lbl = name + "_lbl";
    ObjectCreate(0, lbl, OBJ_TEXT, 0, t2, p2);
    ObjectSetInteger(0, lbl, OBJPROP_COLOR,    col);
    ObjectSetInteger(0, lbl, OBJPROP_FONTSIZE, 9);
    ObjectSetString(0,  lbl, OBJPROP_TEXT, (dir > 0) ? "BOS ↑" : "BOS ↓");
    ObjectSetInteger(0, lbl, OBJPROP_ANCHOR,
                     (dir > 0) ? ANCHOR_RIGHT_UPPER : ANCHOR_RIGHT_LOWER);
   }
```

В этом участке кода мы рисуем все зоны SMC с помощью функции DrawZoneRect, которая создаёт залитый прямоугольник между двумя координатами времени и цены. Внутри прямоугольника размещается выровненная по центру текстовая метка с помощью объекта OBJ_TEXT, привязанного к

средней точке. Текст и цвет меток различаются в зависимости от концепции: зелёный — «Bullish OB», красный — «Bearish OB», синий — «Bullish FVG» и оранжевый — «Bearish FVG».

Уровни слома структуры (BOS) отображаются в виде трендовых линий с помощью функции DrawBOS, при этом метка направления привязывается к точке пробоя. Все функции рисования перед созданием любых объектов проверяют, существуют ли они уже, что предотвращает накопление дубликатов объектов на графике при обработке нескольких событий бара.

```
//+------------------------------------------------------------------+
//|   CREATE PANEL                                                   |
//+------------------------------------------------------------------+
void CreatePanel()
  {
   string bg = "SMC_Panel_BG";
   if(ObjectFind(0, bg) == -1)
     {
      ObjectCreate(0, bg, OBJ_RECTANGLE_LABEL, 0, 0, 0);
      ObjectSetInteger(0, bg, OBJPROP_XDISTANCE,  PanelX);
      ObjectSetInteger(0, bg, OBJPROP_YDISTANCE,  PanelY);
      ObjectSetInteger(0, bg, OBJPROP_XSIZE,      PANEL_W);
      ObjectSetInteger(0, bg, OBJPROP_YSIZE,      PANEL_H);
      ObjectSetInteger(0, bg, OBJPROP_BGCOLOR,    PanelBG);
      ObjectSetInteger(0, bg, OBJPROP_BORDER_TYPE,BORDER_FLAT);
      ObjectSetInteger(0, bg, OBJPROP_COLOR,      PanelAccent);
      ObjectSetInteger(0, bg, OBJPROP_WIDTH,      1);
      ObjectSetInteger(0, bg, OBJPROP_BACK,       false);
      ObjectSetInteger(0, bg, OBJPROP_CORNER,     CORNER_LEFT_UPPER);
     }
  }

void PanelLabel(string name, int line, string text, color clr, int fontsize = 9)
  {
   if(ObjectFind(0, name) == -1)
     {
      ObjectCreate(0, name, OBJ_LABEL, 0, 0, 0);
      ObjectSetInteger(0, name, OBJPROP_CORNER,    CORNER_LEFT_UPPER);
      ObjectSetInteger(0, name, OBJPROP_XDISTANCE, PanelX + 10);
     }
   ObjectSetInteger(0, name, OBJPROP_YDISTANCE, PanelY + 8 + line * PANEL_LN);
   ObjectSetString(0,  name, OBJPROP_TEXT,      text);
   ObjectSetInteger(0, name, OBJPROP_COLOR,     clr);
   ObjectSetInteger(0, name, OBJPROP_FONTSIZE,  fontsize);
   ObjectSetString(0,  name, OBJPROP_FONT,      "Consolas");
  }

//+------------------------------------------------------------------+
//|  UPDATE PANEL                                                    |
//+------------------------------------------------------------------+
void UpdatePanel()
  {
   CreatePanel();

   color accent  = PanelAccent;
   color txtClr  = PanelText;
   color valClr  = clrWhite;

   color strengthColor = (g_trend_str == TS_STRONG) ? clrLimeGreen
```

```
                          : (g_trend_str == TS_MEDIUM) ? clrYellow
                          :                              clrTomato;

   color signalColor   = (g_panel_signal == "Buy")  ? clrLimeGreen
                          : (g_panel_signal == "Sell") ? clrTomato
                          :                              txtClr;

   PanelLabel("SMC_P_title",  0, "  SMC AI Filter v2.1",      accent,  10);
   PanelLabel("SMC_P_sep",    1, StringFormat("%s", "──────────────────────────"), txtCl
   PanelLabel("SMC_P_sig_l",  2, "Current Signal :",          txtClr);

   string sigText = g_panel_signal + "  from  " + g_panel_origin;
   PanelLabel("SMC_P_sig_v",  3, "  " + sigText,              signalColor);

   PanelLabel("SMC_P_str_l",  4, "Trend Strength :",          txtClr);
   PanelLabel("SMC_P_str_v",  5, "  " + StrengthToString(g_trend_str), strengthColor);

   string aiTxt = UseAI
                  ? StringFormat("%.2f", g_ai_score)
                  : "Disabled";
   PanelLabel("SMC_P_ai_l",   6, "AI Score       :",          txtClr);
   PanelLabel("SMC_P_ai_v",   7, "  " + aiTxt,               valClr);

   double _sl, _tp;
   CalcSLTP(_sl, _tp);
   string sltp_mode = (SL_Mode == ATR_MULT) ? "ATR" : "Fixed";
   PanelLabel("SMC_P_sl_l",   8, StringFormat("SL $%.1f  TP $%.1f  [%s]",
              _sl, _tp, sltp_mode),          txtClr);
   string trailTxt = Trail_SL
                     ? StringFormat("ON  trig:$%.1f  step:$%.1f", Trail_TriggerDist, Tr
                     : "OFF";
   color trailClr = Trail_SL ? clrLimeGreen : clrTomato;
   PanelLabel("SMC_P_trail_l",  9, "Trail SL       :",  txtClr);
   PanelLabel("SMC_P_trail_v", 10, "  " + trailTxt,      trailClr);
   PanelLabel("SMC_P_sep2",    11, StringFormat("%s", "──────────────────────────"), txt
   PanelLabel("SMC_P_sym",     12, StringFormat("%-10s  Magic %-6I64d",
              _Symbol, MagicNumber),    txtClr, 8);
   ChartRedraw(0);
  }

void DestroyPanel()
  {
   string names[] =
     {
      "SMC_Panel_BG",
      "SMC_P_title","SMC_P_sep","SMC_P_sig_l","SMC_P_sig_v",
      "SMC_P_str_l","SMC_P_str_v","SMC_P_ai_l","SMC_P_ai_v",
      "SMC_P_sl_l","SMC_P_trail_l","SMC_P_trail_v","SMC_P_sep2","SMC_P_sym"
     };
   for(int i = 0; i < ArraySize(names); i++)
      ObjectDelete(0, names[i]);
  }
```

Панель состоит из одного фонового объекта OBJ_RECTANGLE_LABEL и ряда объектов OBJ_LABEL, расположенных один над другим с фиксированными вертикальными интервалами. Для создания или обновления каждой строки мы используем PanelLabel, которая в рамках одного вызова обрабатывает как первоначальное создание, так и последующие обновления. На панели отображаются текущее

направление и источник сигнала, сила тренда с цветовой кодировкой, оценка уверенности ИИ, текущие значения SL и TP с указанием их режима, а также статус трейлинг-стопа. Зелёный цвет обозначает активные или положительные состояния, а красный — выключенные или слабые состояния. В обработчике OnDeinit функция DestroyPanel аккуратно удаляет все именованные объекты.

```
//+------------------------------------------------------------------+
//|  ORDER BLOCK STRATEGY                                            |
//+------------------------------------------------------------------+
static datetime s_OB_lastDetect = 0;
void DetectAndDrawOrderBlocks()
  {
   datetime lastBar = (datetime)SeriesInfoInteger(_Symbol, _Period, SERIES_LASTBAR_DATE

   //--- Reset OB on new bar
   if(s_OB_lastDetect != lastBar)
     {
      g_OB_valid      = false;
      s_OB_lastDetect = lastBar;
     }

   //--- Scan for a fresh OB candidate
   if(!g_OB_valid)
     {
      for(int i = 1; i < 100; i++)
        {
         //--- Bullish OB
         if(getOpen(i) < getClose(i) &&
            getOpen(i+2) < getClose(i+2) &&
            getOpen(i+3) > getClose(i+3) &&
            getOpen(i+3) < getClose(i+2))
           {
            g_OB.direction = 1;
            g_OB.time      = getTime(i+3);
            g_OB.high      = getHigh(i+3);
            g_OB.low       = getLow(i+3);
            g_OB_valid     = true;
            break;
           }
         //--- Bearish OB
         if(getOpen(i) > getClose(i) &&
            getOpen(i+2) > getClose(i+2) &&
            getOpen(i+3) < getClose(i+3) &&
            getOpen(i+3) > getClose(i+2))
           {
            g_OB.direction = -1;
            g_OB.time      = getTime(i+3);
            g_OB.high      = getHigh(i+3);
            g_OB.low       = getLow(i+3);
            g_OB_valid     = true;
            break;
           }
        }
     }

   if(!g_OB_valid)
      return;
   if(g_lastOBTradeTime == g_OB.time)
```

```
   return; // already traded this signal
if(HasOpenPosition())
   return; // wait for current trade to close

Bid = SymbolInfoDouble(_Symbol, SYMBOL_BID);
Ask = SymbolInfoDouble(_Symbol, SYMBOL_ASK);

bool inBull = (g_OB.direction > 0 && Ask <= g_OB.high && Ask >= g_OB.low);
bool inBear = (g_OB.direction < 0 && Bid >= g_OB.low  && Bid <= g_OB.high);

if(!inBull && !inBear)
   return;

//--- Draw zone rectangle NOW (price within the zone)
datetime tNow = lastBar;
string   rectName = "OB_ZONE_" + TimeToString(g_OB.time);

if(inBull)
  {
   DrawZoneRect(rectName, g_OB.time, tNow, g_OB.high, g_OB.low,
                CLR_BULL_OB, "Bullish OB");
  }
else
  {
   DrawZoneRect(rectName, g_OB.time, tNow, g_OB.high, g_OB.low,
                CLR_BEAR_OB, "Bearish OB");
  }

//--- Find most-recent swing H and L
double   bestH = 0, bestL = 0;
datetime bestHT = 0, bestLT = 0;

for(int i = 0; i < 50; i++)
  {
   fib_high = 0;
   fib_low = 0;
   fib_t1 = 0;
   fib_t2 = 0;
   DetectSwingForBar(i, SWING_OB);
   if(fib_high > 0 && (bestHT == 0 || fib_t1 > bestHT))
     {
      bestH = fib_high;
      bestHT = fib_t1;
     }
   if(fib_low  > 0 && (bestLT == 0 || fib_t2 > bestLT))
     {
      bestL = fib_low;
      bestLT = fib_t2;
     }
  }
if(bestHT == 0 || bestLT == 0)
   return;

//--- Fibonacci validation
if(inBull)
  {
   double entLvl = bestH - (bestH - bestL) * (Fib_Trade_lvls / 100.0);
```

```
      if(Ask > entLvl)
         return; // not deep enough

      //--- AI filter
      g_ai_score = RunONNX(0, 1, g_OB.high, g_OB.low, Ask);
      if(UseAI && g_ai_score < AI_MinScore)
        {
         Print("[AI] OB Bull signal rejected - score: ", DoubleToString(g_ai_score, 3))
         return;
        }

      g_trend_str = ScoreToStrength(g_ai_score);
      ExecuteTrade(ORDER_TYPE_BUY, "OB");
      g_lastOBTradeTime = g_OB.time;
      g_OB_valid = false;
     }
   else // inBear
     {
      double entLvl = bestL + (bestH - bestL) * (Fib_Trade_lvls / 100.0);
      if(Bid < entLvl)
         return;

      g_ai_score = RunONNX(0, -1, g_OB.high, g_OB.low, Bid);
      if(UseAI && g_ai_score < AI_MinScore)
        {
         Print("[AI] OB Bear signal rejected - score: ", DoubleToString(g_ai_score, 3))
         return;
        }

      g_trend_str = ScoreToStrength(g_ai_score);
      ExecuteTrade(ORDER_TYPE_SELL, "OB");
      g_lastOBTradeTime = g_OB.time;
      g_OB_valid = false;
     }
  }
```

Стратегия ордер-блоков (OB) просматривает недавние бары назад в поисках классического трёхсвечного паттерна, который определяет бычий или медвежий ордер-блок. Бычий ордер-блок (OB) — это медвежья свеча, за которой следует бычий импульс. Медвежий ордер-блок (OB) — это обратная ситуация. Как только обнаруживается валидный OB, он сохраняется и повторно используется при последующих событиях бара, пока по нему не будет открыта сделка или он не будет признан недействительным. Когда цена откатывается в зону OB, мы рисуем цветной прямоугольник, проверяем глубину отката с помощью уровня Фибоначчи, а затем запускаем фильтр ИИ. Если оценка проходит фильтр, сделка открывается. Время ордер-блока (OB) фиксируется в переменной g_lastOBTradeTime, чтобы гарантировать, что по одной и той же зоне никогда не будет открыта сделка дважды, даже если цена вернётся к ней на более позднем баре.

```
void DetectAndDrawFVGs()
  {
   double point = SymbolInfoDouble(_Symbol, SYMBOL_POINT);
   int    maxBars = MathMin(FVG_ScanBars, Bars(_Symbol, _Period)) - 2;

   Bid = SymbolInfoDouble(_Symbol, SYMBOL_BID);
   Ask = SymbolInfoDouble(_Symbol, SYMBOL_ASK);

   datetime barNow = (datetime)SeriesInfoInteger(_Symbol, _Period, SERIES_LASTBAR_DATE)
```

```
   if(OneTradePerBar && lastFVGTradeBar == barNow)
      return;
   if(HasOpenPosition())
      return; // wait for current trade to close

   for(int i = 2; i < maxBars; i++)
     {
      double lowA  = getLow(i+2),  highA = getHigh(i+2);
      double highC = getHigh(i),   lowC  = getLow(i);

      SFVG z;
      bool found = false;

      if(lowA > highC && (lowA - highC) >= FVG_MinPoints * point)
        {
         z.dir = 1;
         z.tLeft = getTime(i+2);
         z.top = lowA;
         z.bot = highC;
         found = true;
        }
      else
         if(highA < lowC && (lowC - highA) >= FVG_MinPoints * point)
           {
            z.dir = -1;
            z.tLeft = getTime(i+2);
            z.top = lowC;
            z.bot = highA;
            found = true;
           }

      if(!found)
         continue;

      double mid = (z.top + z.bot) * 0.5;
      bool inGap = false;

      if(z.dir == 1)
         inGap = (Ask <= z.top && Ask >= z.bot && (!FVG_TradeAtEQ || Ask <= mid));
      else
         inGap = (Bid <= z.top && Bid >= z.bot && (!FVG_TradeAtEQ || Bid >= mid));

      if(!inGap)
         continue;

      //--- Draw coloured zone only when price enters
      DrawFVGZone(z);

      //--- AI filter
      double entry = (z.dir == 1) ? Ask : Bid;
      g_ai_score = RunONNX(1, z.dir, z.top, z.bot, entry);
      if(UseAI && g_ai_score < AI_MinScore)
        {
         Print("[AI] FVG signal rejected - score: ", DoubleToString(g_ai_score, 3));
         continue;
        }
```

```
        g_trend_str = ScoreToStrength(g_ai_score);

        if(z.dir == 1)
           ExecuteTrade(ORDER_TYPE_BUY,  "FVG");
        else
           ExecuteTrade(ORDER_TYPE_SELL, "FVG");

        lastFVGTradeBar = barNow;
        break;
       }
   }
```

Для FVG мы просматриваем последние бары в поисках дисбаланса из трёх свечей. Бычий гэп справедливой стоимости (FVG) существует, когда минимум первой свечи находится выше максимума третьей. Медвежий гэп справедливой стоимости (FVG) — это зеркальное условие. Границы гэпа определяют зону, а середина — уровень равновесия, используемый при включённом параметре FVG_TradeAtEQ. Когда цена входит в гэп и выполняет условие EQ, мы рисуем цветной прямоугольник зоны и запускаем инференс ИИ. При бычьем FVG рисуется синий прямоугольник с меткой «Bullish FVG», а при медвежьем FVG — оранжевый прямоугольник с меткой «Bearish FVG». Затем мы фиксируем время бара в переменной lastFVGTradeBar, чтобы предотвратить несколько входов в рамках одного и того же бара.

```
//+------------------------------------------------------------------+
//|  BOS STRATEGY                                                    |
//+------------------------------------------------------------------+
void DetectAndDrawBOS()
  {
   double   bestH = -1, bestL = -1;
   datetime bestHT = 0, bestLT = 0;

   for(int i = 0; i < 50; i++)
     {
      swng_High = 0;
      swng_Low = 0;
      bos_tH = 0;
      bos_tL = 0;
      DetectSwingForBar(i, SWING_BOS);

      if(swng_High > 0 && (bestHT == 0 || bos_tH > bestHT))
        { bestH = swng_High; bestHT = bos_tH; }
      if(swng_Low > 0 && (bestLT == 0 || bos_tL > bestLT))
        { bestL = swng_Low;  bestLT = bos_tL; }
     }

   if(bestHT > 0)
     {
      if(bos_tH != bestHT)
         Bull_BOS_traded = false;
      swng_High = bestH;
      bos_tH = bestHT;
     }
   if(bestLT > 0)
     {
      if(bos_tL != bestLT)
         Bear_BOS_traded = false;
      swng_Low = bestL;
```

```
      bos_tL = bestLT;
    }

  Bid = SymbolInfoDouble(_Symbol, SYMBOL_BID);
  Ask = SymbolInfoDouble(_Symbol, SYMBOL_ASK);
  datetime currentBar = getTime(0);

  if(HasOpenPosition())
     return; // wait for current trade to close

  //--- BUY on break above swing high
  if(swng_High > 0 && Ask > swng_High && !Bull_BOS_traded)
    {
     if(lastBOSTradeTime != currentBar || lastBOSTradeDirection != 1)
       {
        g_ai_score = RunONNX(2, 1, swng_High + 5 * _Point, swng_High - 5 * _Point, Ask
        if(!UseAI || g_ai_score >= AI_MinScore)
          {
           if(DrawBOSLines)
              DrawBOS("BOS_H_" + TimeToString(bos_tH), bos_tH, swng_High,
                      TimeCurrent(), swng_High, CLR_BOS_BULL, 1);
           g_trend_str = ScoreToStrength(g_ai_score);
           ExecuteTrade(ORDER_TYPE_BUY, "BOS");
           lastBOSTradeTime      = currentBar;
           lastBOSTradeDirection = 1;
           Bull_BOS_traded       = true;
           swng_High             = -1.0;
          }
        else
           Print("[AI] BOS Bull rejected — score: ", DoubleToString(g_ai_score, 3));
       }
    }

  //--- SELL on break below swing low
  if(swng_Low > 0 && Bid < swng_Low && !Bear_BOS_traded)
    {
     if(lastBOSTradeTime != currentBar || lastBOSTradeDirection != -1)
       {
        g_ai_score = RunONNX(2, -1, swng_Low + 5 * _Point, swng_Low - 5 * _Point, Bid)
        if(!UseAI || g_ai_score >= AI_MinScore)
          {
           if(DrawBOSLines)
              DrawBOS("BOS_L_" + TimeToString(bos_tL), bos_tL, swng_Low,
                      TimeCurrent(), swng_Low, CLR_BOS_BEAR, -1);
           g_trend_str = ScoreToStrength(g_ai_score);
           ExecuteTrade(ORDER_TYPE_SELL, "BOS");
           lastBOSTradeTime      = currentBar;
           lastBOSTradeDirection = -1;
           Bear_BOS_traded       = true;
           swng_Low              = -1.0;
          }
        else
           Print("[AI] BOS Bear rejected — score: ", DoubleToString(g_ai_score, 3));
       }
    }
}
```

```
//+------------------------------------------------------------------+
```

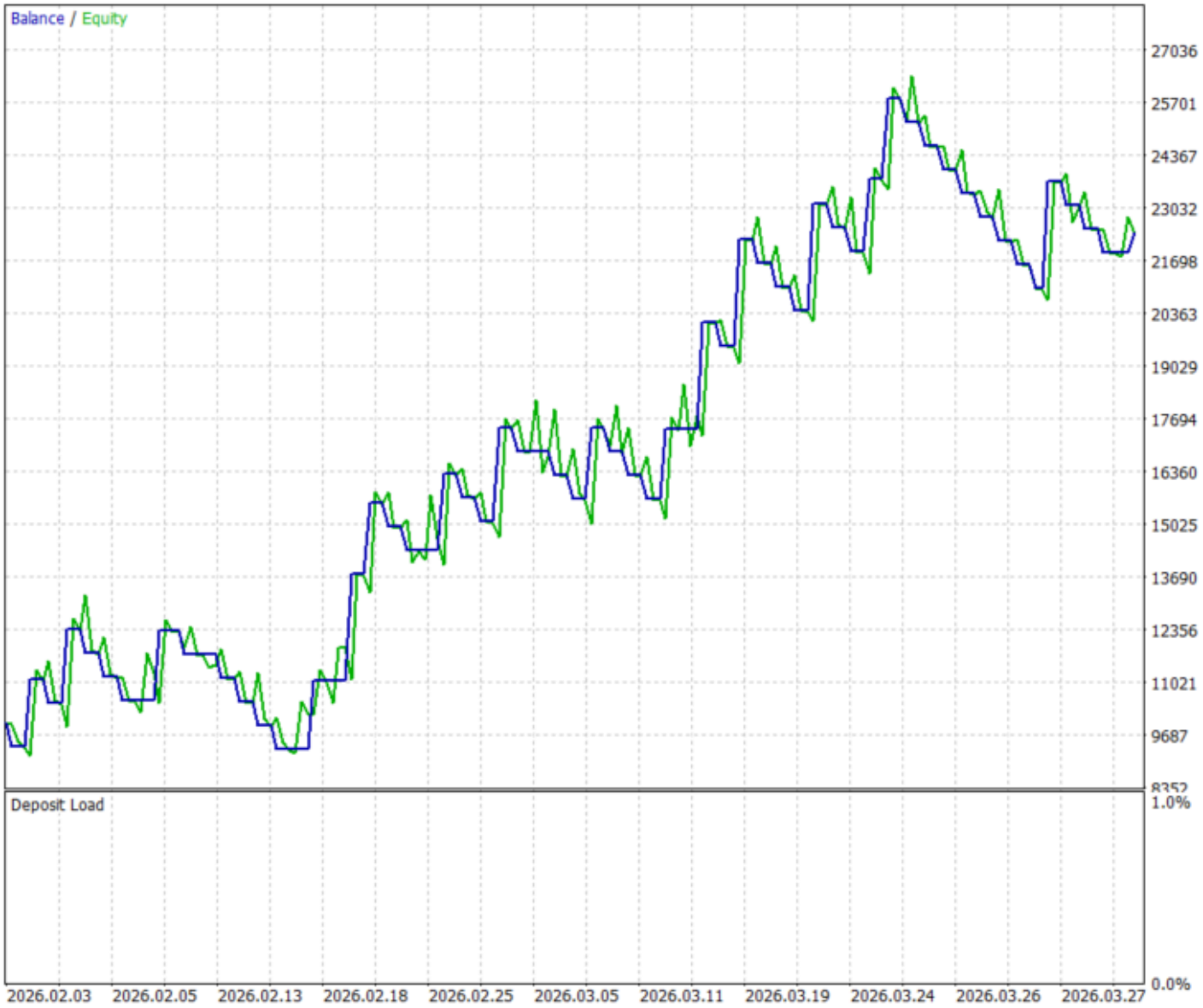
Наконец, мы определяем последние свинговый максимум и свинговый минимум, просматривая последние пятьдесят баров с помощью функции DetectSwingForBar. Когда цена пробивает свинговый максимум вверх, мы рассматриваем это как бычий слом структуры (BOS) и открываем сделку на покупку. Когда цена пробивает свинговый минимум вниз, мы рассматриваем это как медвежий слом структуры (BOS) и открываем сделку на продажу. Каждый пробой отображается в виде горизонтальной трендовой линии на пробитом уровне с меткой «BOS ↑» или «BOS ↓». Флаги Bull_BOS_traded и Bear_BOS_traded предотвращают ситуацию, при которой один и тот же слом структуры приводит к открытию более одной сделки. Как только по свинговому уровню открыта сделка, он сбрасывается, чтобы стратегия могла определить следующую свежую структуру по мере дальнейшего развития цены.

Результаты бэктеста

Бэктест проводился на примерно двухмесячном окне тестирования с 2 февраля 2026 года по 1 апреля 2026 года со следующими настройками:

Variable	Value
=== Strategy ===	
TradeStrategy	STRAT_AUTO
In_Lot	0.02
MagicNumber	76543
=== SL / TP ===	
SL_Mode	PRICE_DIST
SL in price units ($)	30
TP in price units ($)	135
SL = ATR(14) × this	1.5
TP = ATR(14) × this	3
Trail_SL	true
profit ($) needed to start trailing	99
trail step ($) — SL moves in this increment	14.5
=== SMC Parameters ===	
bars each side for swing	5
OB fib retrace %	61.8
DrawBOSLines	true
FVG_MinPoints	3
FVG_ScanBars	20
FVG_TradeAtEQ	true
OneTradePerBar	true
=== ONNX AI Filter ===	
UseAI	true
min probability to allow trade	0.6
threshold for "Strong" trend rating	0.8
threshold for "Medium" trend rating	0.65

Ниже приведены кривая капитала и результаты бэктеста:



History Quality	100%				
Bars	964	Ticks	14745010	Symbols	1
Initial Deposit	10 000.00				
Total Net Profit	12 412.18	Balance Drawdown Ab...	671.70	Equity Drawdown Abso...	787.46
Gross Profit	34 657.86	Balance Drawdown Ma...	4 807.04 (18.6...	Equity Drawdown Maxi...	5 675.24 (21.48...
Gross Loss	-22 245.68	Balance Drawdown Rel...	24.56% (3 037....	Equity Drawdown Relat...	30.13% (3 996....
Profit Factor	1.56	Expected Payoff	229.86	Margin Level	46062700.00%
Recovery Factor	2.19	Sharpe Ratio	5.18	Z-Score	0.70 (51.61%)
AHPR	1.0185 (1.85%)	LR Correlation	0.93	OnTester result	0
GHPR	1.0151 (1.51%)	LR Standard Error	1 830.97		
Total Trades	54	Short Trades (won %)	10 (50.00%)	Long Trades (won %)	44 (27.27%)
Total Deals	108	Profit Trades (% of total)	17 (31.48%)	Loss Trades (% of total)	37 (68.52%)
	Largest	profit trade	2 702.66	loss trade	-609.12
	Average	profit trade	2 038.70	loss trade	-601.23
	Maximum	consecutive wins ($)	3 (6 253.40)	consecutive losses ($)	8 (-4 807.04)
	Maximal	consecutive profit (cou...	6 253.40 (3)	consecutive loss (count)	-4 807.04 (8)
	Average	consecutive wins	1	consecutive losses	3

Заключение

В этой статье мы с нуля создали полноценную торговую систему, дополненную ИИ. Мы начали с пайплайна на Python, который извлекает исторические данные по инструменту XAUUSD на таймфрейме H1 из MetaTrader 5 и реконструирует события ордер-блоков (OB), гэпов справедливой стоимости (FVG) и сломов структуры (BOS) с использованием логики советника. Каждое событие было размечено на основе его торгового результата. Это позволило нам сконструировать двенадцать нормализованных признаков для геометрии зон, импульса и времени торговой сессии. Затем эти признаки использовались для обучения классификатора XGBoost с применением кросс-валидации, после чего модель была экспортирована в формат ONNX и скомпилирована в советник как встроенный ресурс. Со стороны MQL5 советник был переработан так, чтобы вычислять вектор признаков в момент появления сигнала, оценивать его с помощью модели ИИ и фильтровать каждую сделку по возвращаемой оценке уверенности.

Кроме того, мы реализовали расчёт уровней SL и TP с учётом волатильности, используя риск-менеджмент в долларах вместо фиксированных расстояний в пунктах. Защитный механизм «одна сделка на один сигнал» предотвращает повторные входы по одному и тому же сетапу, а трейлинг-стоп на уровне тиков постепенно фиксирует прибыль по мере движения цены. На панели графика в режиме реального времени отображаются источник сигнала, оценка уверенности ИИ и сила тренда, что даёт трейдеру понятную обратную связь во время исполнения сделки.

После прочтения этой статьи трейдер получает полный пайплайн обучения на Python и советник MQL5 с поддержкой ИИ, готовый к тестированию или запуску в реальных условиях. Кроме того, система создаёт прочную основу для будущей оптимизации и проведения экспериментов. Самое главное, в статье показано, как машинное обучение может усилить Smart Money Concepts за счёт объективного отбора сигналов и более дисциплинированного, основанного на данных процесса принятия решений.

Ниже приводится краткое описание содержимого архива «MQL5 zip»:

Имя файла	Описание файла
SMC_AI_NoteBK.ipynb	Jupyter-ноутбук для обучения модели ONNX, основанный на исходной логике советника.
smc_filter.onnx	Обученная модель ONNX.
SMC AI.mq5	Это советник MetaTrader 5, работающий с обученной моделью ONNX.

Следующая статья >>

Hlomohang John Borotho

Founder and CEO в Dynamic Financial Labs Лесото 15871

Quantitative intraday trading strategies
· MQL5 Expert Advisor development ·
Python + AI automation for MetaTrader 5.
Built by a developer who actually trades.

Другие статьи автора

- Теория графов: Алгоритм Форда—Фалкерсона для оценки ликвидности и пропускной способности рынка

- Интеграция MQL5 с пакетами обработки данных (Часть 9): Адаптивная оценка волатильности на основе энтропии
- Теория графов: Применение алгоритма эвристического поиска A* в торговле
- Автоматизация индикатора рыночной энтропии: Торговая система на основе теории информации
- Разработка динамического мультивалютного советника (Часть 8): Ротация капитала в зависимости от времени суток
- Свинговые экстремумы и откаты в MQL5 (Часть 3): Определение структурной валидности за рамками простых максимумов и минимумов

Подпишитесь и узнавайте о
популярных статьях и кодах